KB금융그룹 | 국민의 평생 금융파트너

2026년 상반기 K-디지털 트레이닝

UI/UX 특강

[KB] IT’s Your Life

KB국민은행

ⓒ2026.정수아.All rights reserved.

- **수업 로드맵**

STEP 1

- UI/UX 이해 및 핵심 원칙
- 좋은 UX를 만드는 기준
- UI 구조 설계 흐름

STEP 2

- 로그인 화면 구조 분석
- 입력폼 및 버튼 역할 분석

STEP 3

- Flexbox, Grid 레이아웃 구현
- 반응형 UI 적용
- CSS 프레임워크 활용 전략

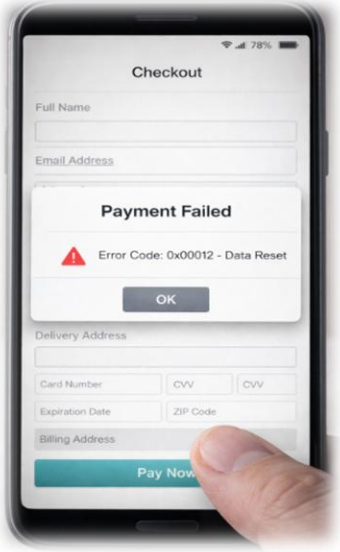
ⓒ2026.정수아.All rights reserved.

오늘 여러분의 기분은 어떤 앱이 결정했나요?

알람

광고

메뉴



에러

ⓒ2026.정수아.All rights reserved.

Image generated by AI

- **사용자를 이탈시키는 4가지 장벽**

장벽	Mobile	Web
조작 실수	알람 '중단' 오클릭	결제 중 '취소' 버튼 오클릭
흐름 단절	도착 정보를 가리는 광고	뉴스 기사 내용을 가리는 광고
정보 과잉	너무 많은 메뉴 카테고리	복잡하고 깊은 메뉴 구조
노력 낭비	에러로 인한 데이터 삭제	전송 실패로 인한 글 삭제

ⓒ2026.정수아.All rights reserved.

사용자 중심 사고

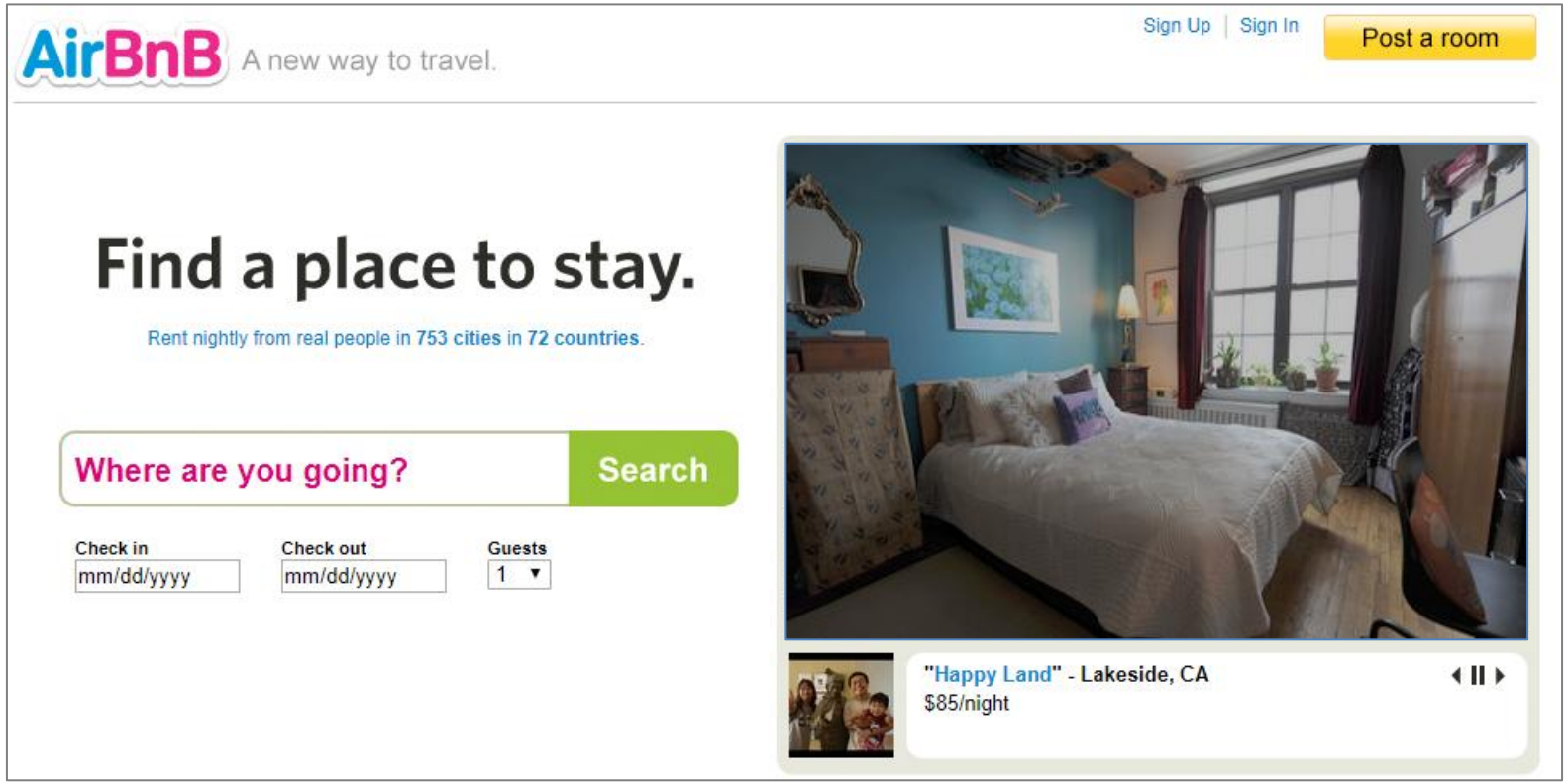
- 에어비앤비 사례로 보는 UI/UX가 만드는 혁신적 변화

airbnb

ⓒ2026.정수아.All rights reserved.

사용자 중심 사고

- 에어비앤비의 위기: 기술은 완벽한데, 왜 아무도 예약하지 않을까?



ⓒ2026.정수아.All rights reserved.

사용자 중심 사고

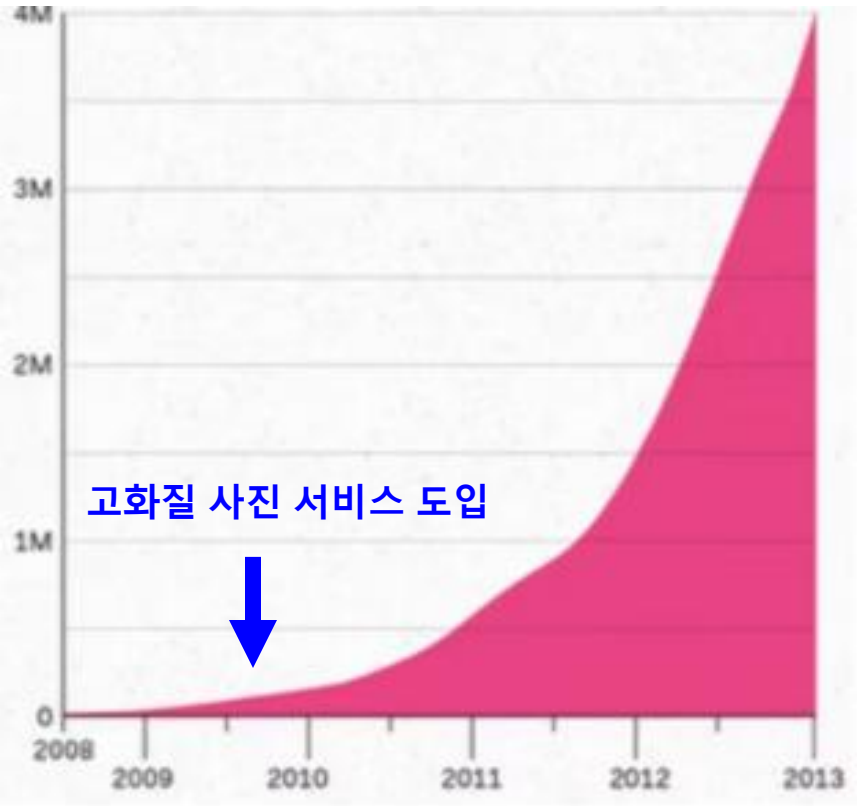
- 에어비앤비의 위기: 기술은 완벽한데, 왜 아무도 예약하지 않을까?

ⓒ2026.정수아.All rights reserved.

Image source : Unsplash

사용자 중심 사고

- 에어비앤비의 반전: 코드가 아닌 '신뢰'를 설계하다



에어비앤비 누적 게스트 수 추이

ⓒ2026.정수아.All rights reserved.

Image source : Unsplash, Airbnb "2012 Year in Review"

사용자 중심 사고

- 에어비앤비의 현재: 100조 가치를 만든 '경험의 힘'

8M+ active listings worldwide	**150K+** cities and towns with active Airbnb listings	**220+** countries and regions with Airbnb listings	**2B+** Airbnb guest arrivals all-time
5M+ hosts on Airbnb	**$300B+** earned by hosts, all-time	**$15K** earned by the typical US host in 2024	**$16B+** total taxes collected and remitted globally

Last updated November 2025

ⓒ2026.정수아.All rights reserved.

사용자 중심 사고

- 토스 사례로 보는 UI/UX가 만드는 혁신적 변화

ⓒ2026.정수아.All rights reserved.

사용자 중심 사고

- 토스의 도전: 우리가 당연하게 여겼던 '불편함'

ⓒ2026.정수아.All rights reserved.

사용자 중심 사고

- 토스의 혁신: 10분을 10초로 줄인 '경험의 마법'

ⓒ2026.정수아.All rights reserved.

Image source : toss 홈페이지

사용자 중심 사고

- 토스의 결과: 금융의 문법을 바꾼 UI/UX의 위력"

ⓒ2026.정수아.All rights reserved.

Image source : toss 홈페이지

사용자 중심 사고

- 마찰 없는 경험: 사용자를 1초도 멈추게 하지 않는 것

FRICTION

FRICTIONLESS

ⓒ2026.정수아.All rights reserved.

Image generated by AI

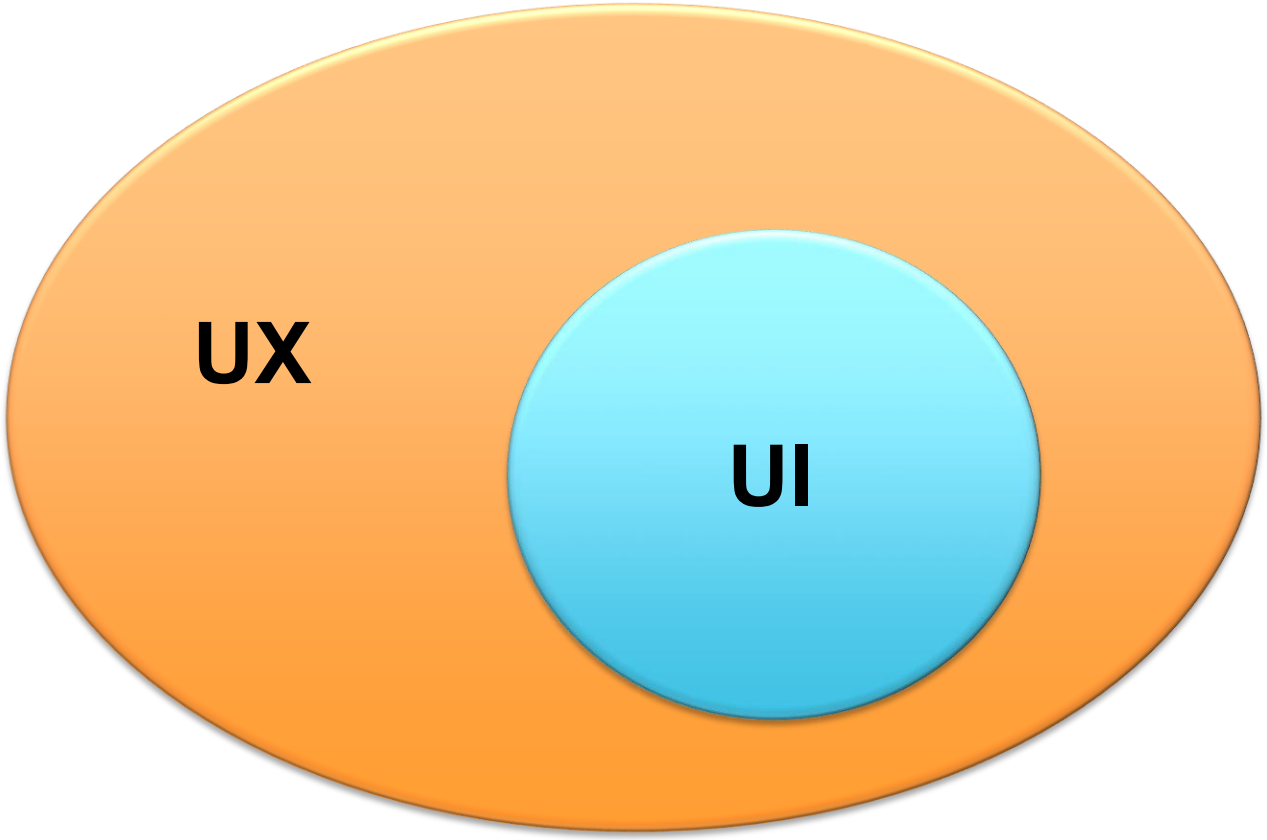
UX 핵심 원칙

- **UX(User Experience)**
  - 사용자가 서비스를 이용하면서 겪는 전체 경험
    - 이해하기 쉬운가
    - 흐름이 자연스러운가
    - 실수하기 쉬운가

- **UI(User Interface)**
  - 사용자가 서비스를 조작하는 화면과 인터페이스
    - 버튼
    - 메뉴
    - 화면 구성
    - 시각 디자인

ⓒ2026.정수아.All rights reserved.

- UI와 UX의 관계

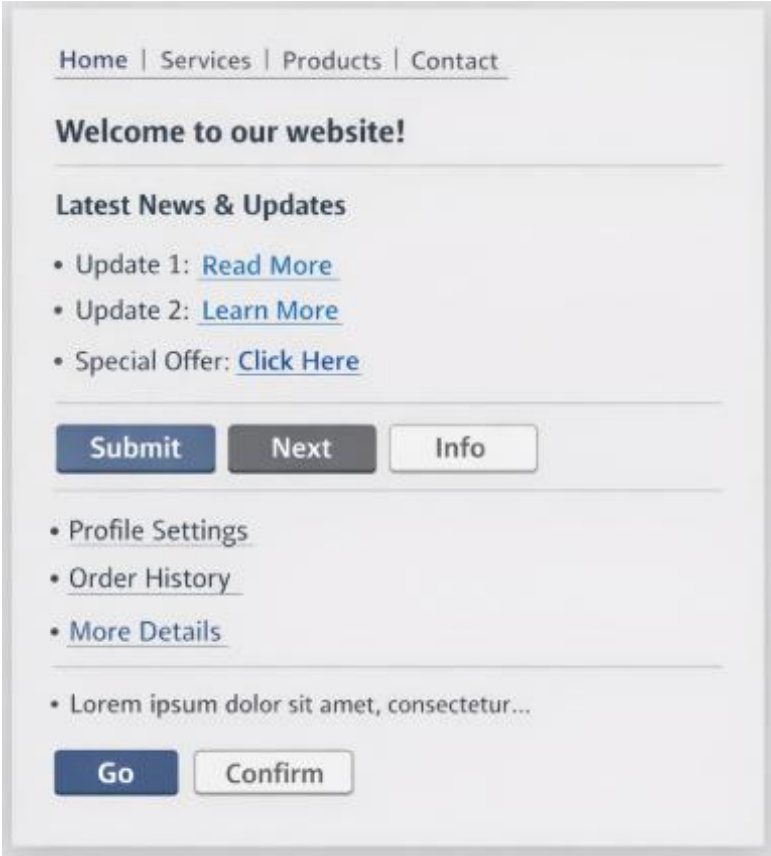


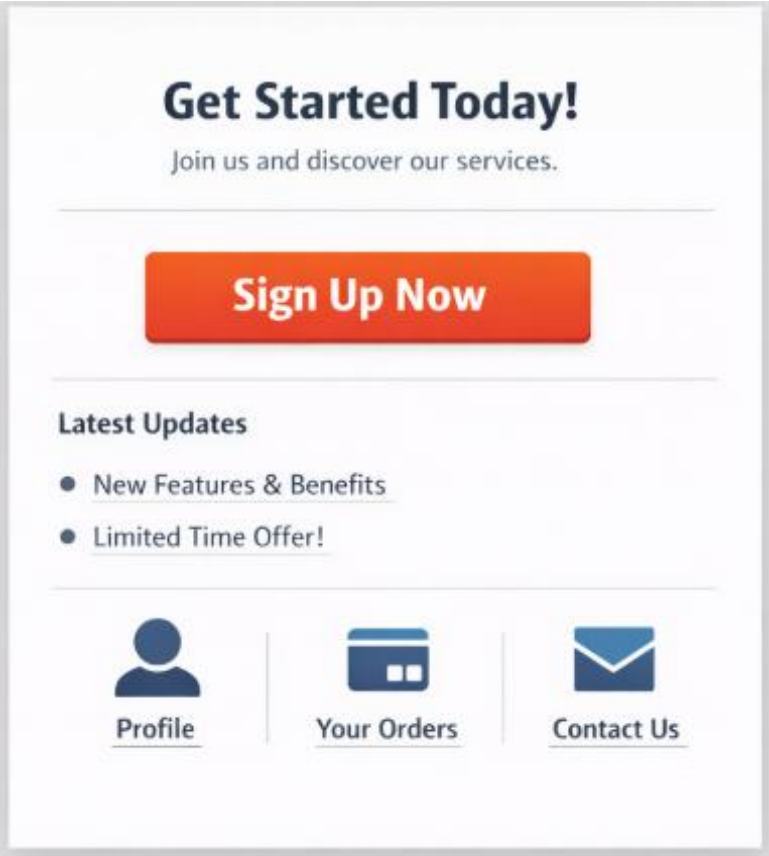
UX는 전체 경험, UI는 그 경험을 구성하는 하나의 요소

ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- 명확성(Clarity) : 고민하는 순간 사용자는 떠난다





ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

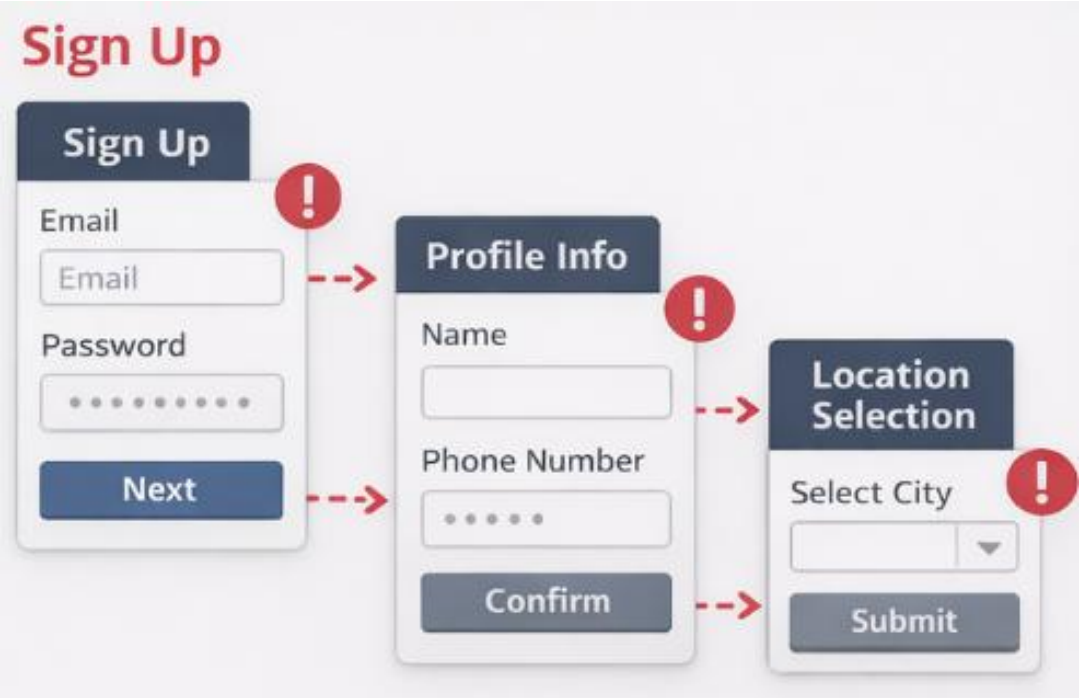
- 흐름성(Flow) : 사용자가 멈추지 않게 만드는 UX

ⓒ2026.정수아.All rights reserved.

Image generated by AI

UX 핵심 원칙

- 흐름성(Flow) : 사용자가 멈추지 않게 만드는 UX



너무 많은 단계

“도대체 언제 끝나는 거지?”

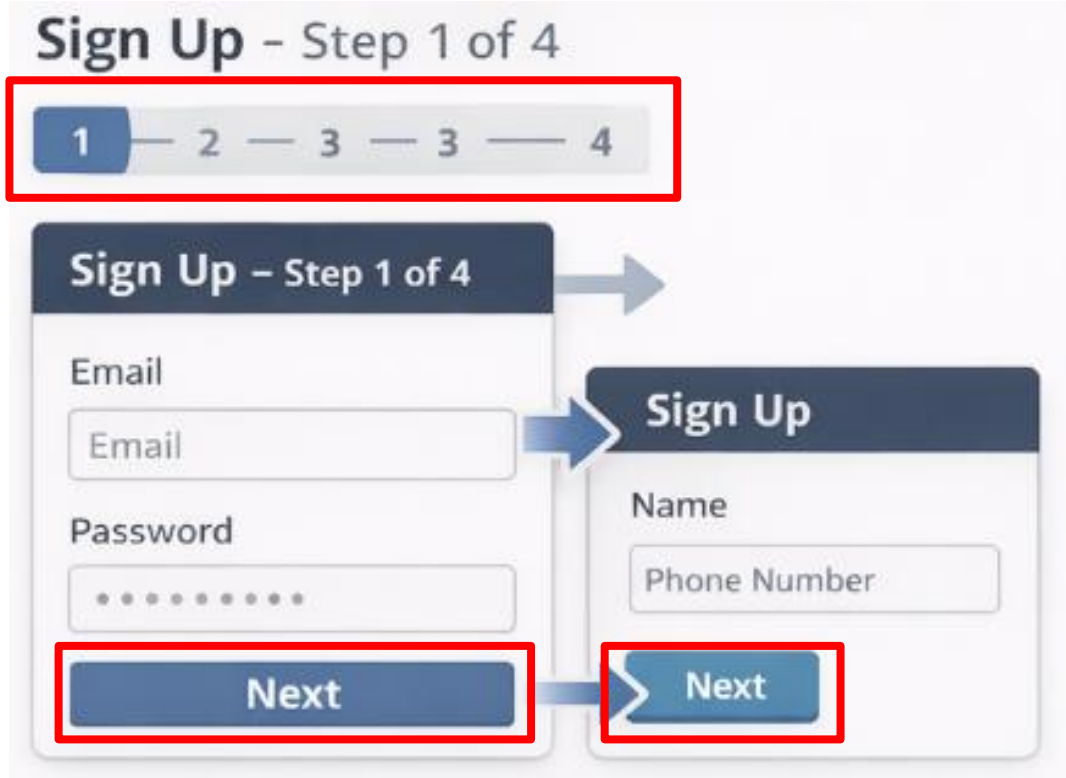
집중력 저하

과정 자체가 부담스러움

ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- 흐름성(Flow) : 사용자가 멈추지 않게 만드는 UX



예측 가능한 단계

안도감을 주는 설계

일관된 행동 유도

생각하지 않아도 되는 흐름

ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- 흐름성(Flow) : 사용자가 멈추지 않게 만드는 UX

개발자의 작은 선택들

버튼 이름

페이지 분할 기준

다음 단계 전환 방식

로딩 후 반응 처리

→

사용자의 경험

확신

예측

연속성

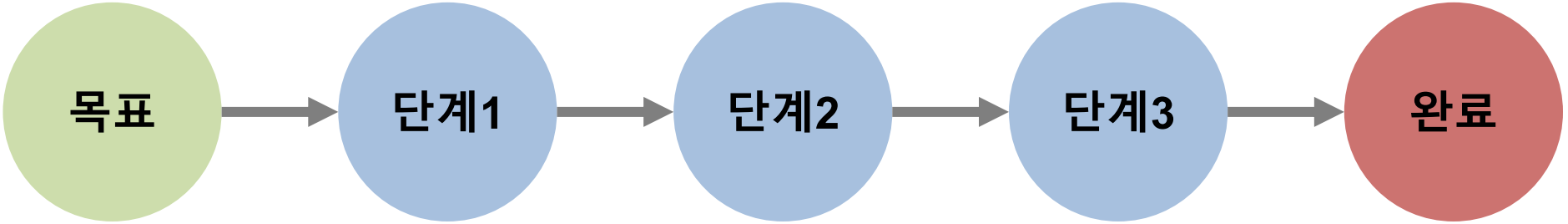
신뢰

ⓒ2026.정수아.All rights reserved.

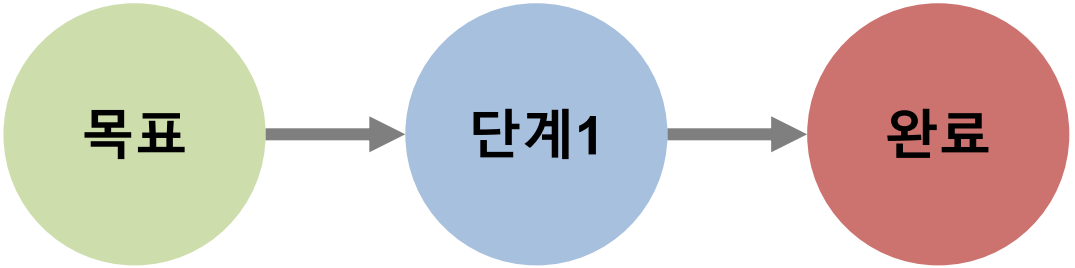
UX 핵심 원칙

- 효율성(Efficiency) : 사용자의 시간을 낭비하지 않는 UX

비효율적인 UX



효율적인 UX



사용자는 항상 가장 빠른 길을 선택한다

ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- **효율성이 중요한 이유**
  - 사용자는 기다리는 것을 싫어한다.
  - 사용자는 반복 입력을 싫어한다.
  - 사용자는 불필요한 단계를 싫어한다.

- **이 단계를 정말로 사용자가 거쳐야 할까?**
  - 좋은 UX 설계의 출발점

ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- **효율적인 UX 설계**
  - 불필요한 단계를 제거한다.
  - 사용자의 입력을 최소화한다.
  - 목표까지의 경로를 짧게 만든다.

ⓒ2026.정수아.All rights reserved.

사용자는 기능을 평가하지 않습니다

과정에서 느낀 경험으로
서비스를 판단합니다

그 경험을 만드는 핵심은 바로 페이지 분할입니다

ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- **페이지, 몇 장으로 나눌까?**
  - 한 페이지에 정보가 너무 많지 않게
  - 3단계가 좋다
  - 5단계는 많다

사용자는 페이지 수를 세지 않습니다

ⓒ2026.정수아.All rights reserved.

- 페이지 수보다 중요한 '예측 가능성'

이게 끝이 보이는 과정인가?

"아, 거의 끝이구나"

"어? 아직도 남았어?"

예측 가능성의 차이가 사용자 경험의 차이를 만든다

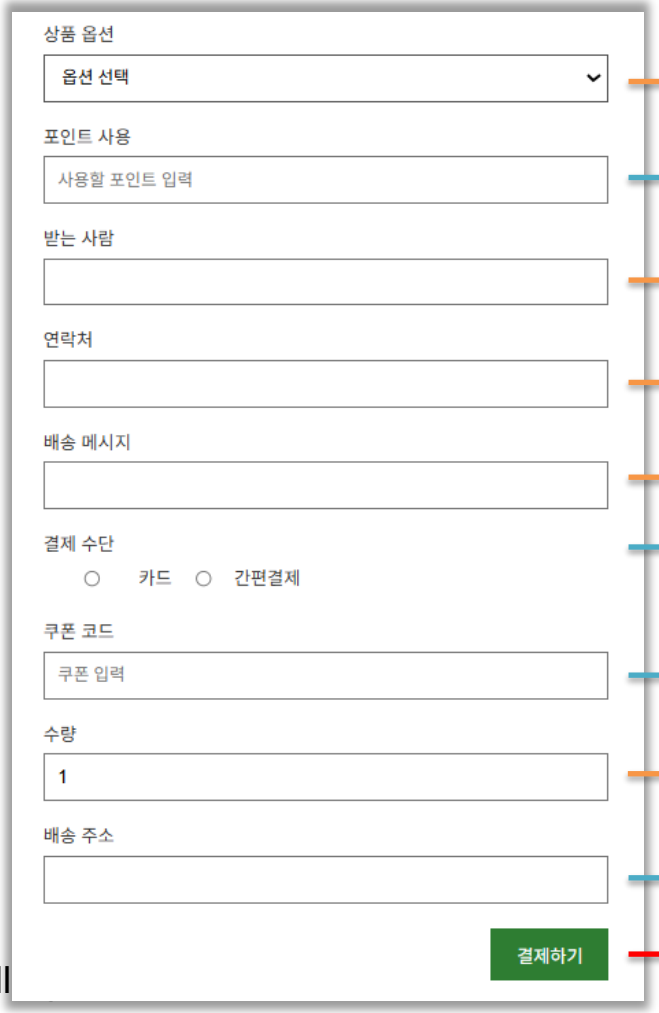
ⓒ2026.정수아.All rights reserved.

- **페이지 분할의 3가지 기준**

기준	설명
논리적 그룹핑(Logical Grouping)	성격이 비슷한 정보끼리 묶는다
인지적 부하 조절 (Cognitive Load Control)	한 화면에서 생각해야 할 양을 줄인다
데이터 의존성 (Data Dependency)	앞 단계의 정보가 다음 단계의 기준이 된다

ⓒ2026.정수아.All rights reserved.

- **논리적 그룹핑**
  - 사용자가 **같은 맥락의 정보**라고 인식하는 것들을 한 페이지에 묶는다.



"아, 내가 살 상품을 고르는 단계구나"

"아직 상품도 다 안 골랐는데?"

"이제 배송 정보인가?"

"아직 주소도 안 썼는데?"

"아직 상품 단계가 안 끝난건가?"

"지금 이게 몇 번째 단계지?"

이 버튼을 눌러도 되는 게 맞나?

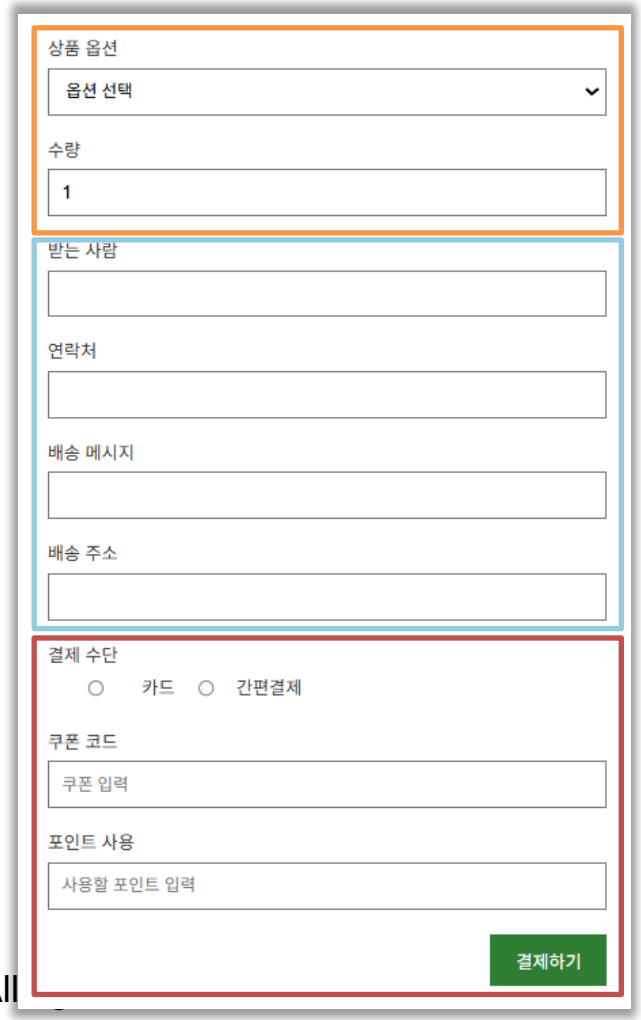
ⓒ2026.정수아.All

- **논리적 그룹핑**
  - 사용자가 **같은 맥락의 정보**라고 인식하는 것들을 한 페이지에 묶는다.

상품

배송

결제



행동 흐름 명확화

인지 부담 감소

단계별 맥락 인식

심리적 안정감

ⓒ2026.정수아.All

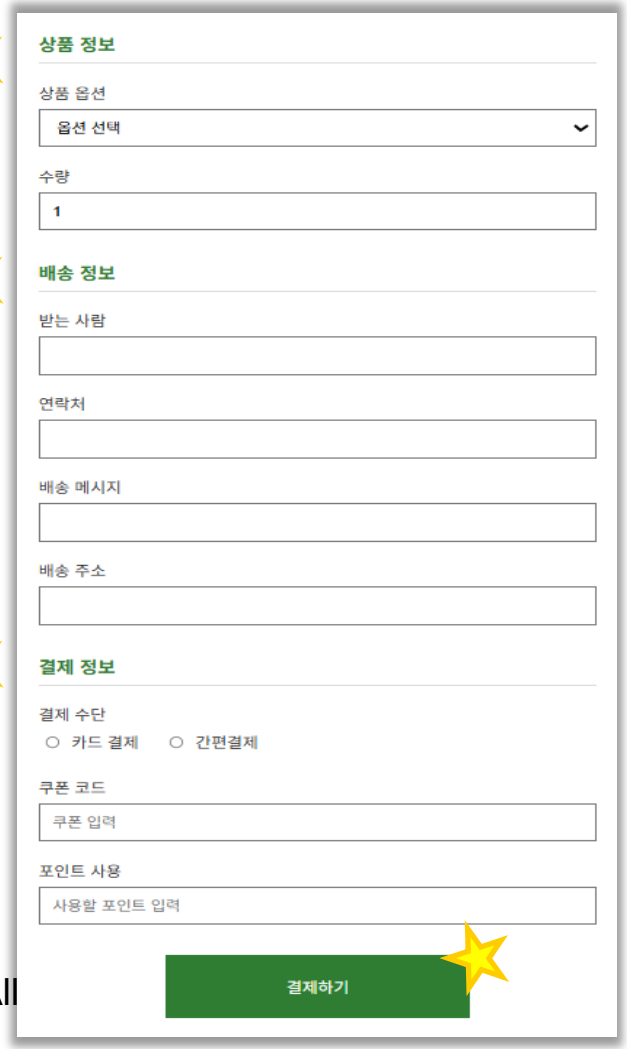
- **인지적 부하 조절**
  - 사용자가 **한 번에 생각**해야 할 정보의 양을 줄인다.

논리적 그룹핑 결과

ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- **인지적 부하 조절**
  - 사용자가 **한 번에 생각**해야 할 정보의 양을 줄인다.



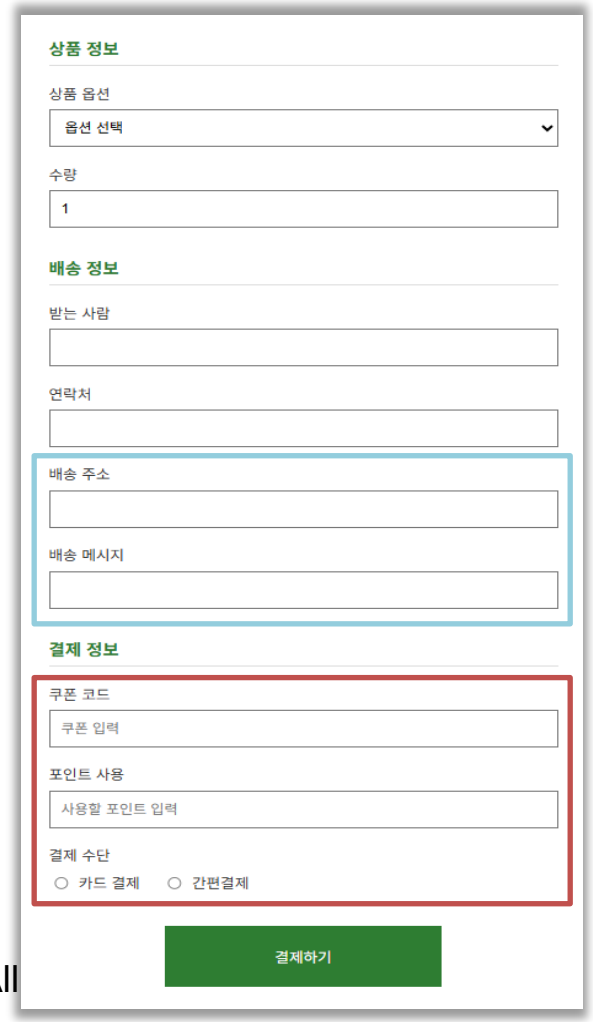
영역 간 간격 조정

단계별 제목 추가

결제 버튼 크기 확대

ⓒ2026.정수아.All

- **데이터 의존성**
  - **이전 선택이나 정보**가 다음 화면의 이해와 결정을 좌우한다.



배송 정보 입력 순서 조정

할인 적용 후 결제 수단 선택

데이터 흐름 기반 순서 정리

ⓒ2026.정수아.All

UX 핵심 원칙

- **안정성(Stability) : 사용자가 안심하고 사용할 수 있는 UX**
  - 안정성을 높이는 설계
    - 예측 가능한 인터페이스
    - 실수 방지 설계
    - 명확한 상태 전달
    - 오류 발생 시 명확한 안내

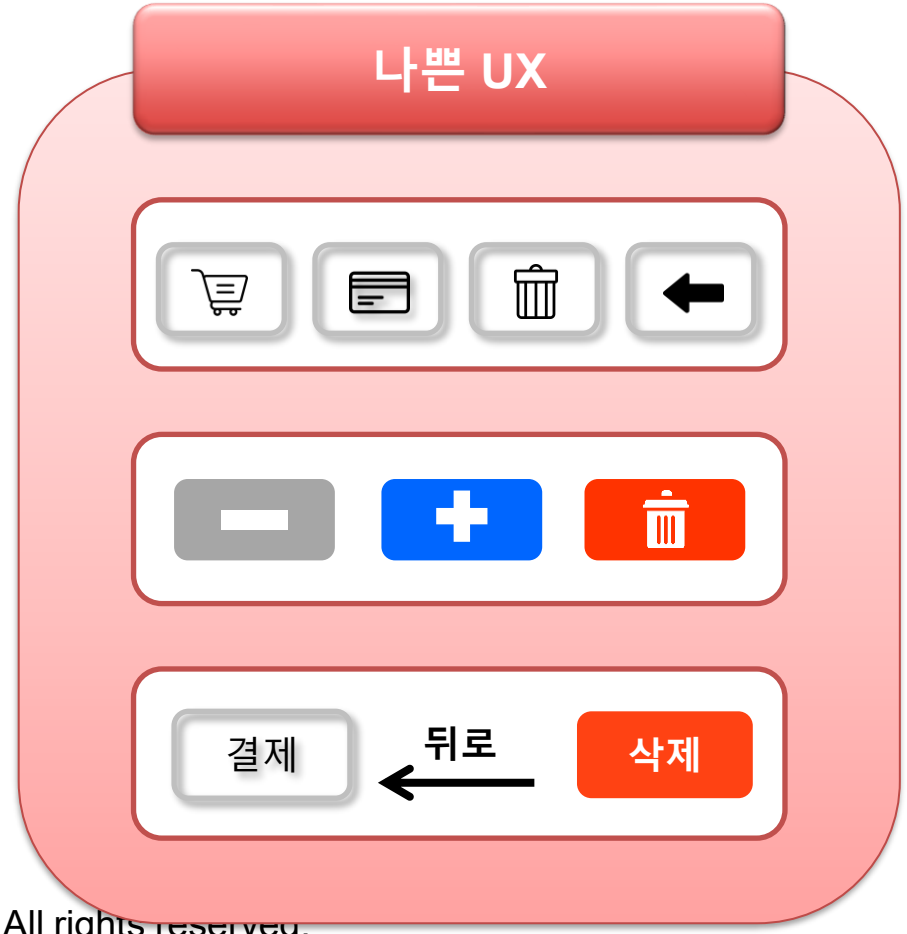
ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- **안정성(Stability) : 사용자가 안심하고 사용할 수 있는 UX**
  - 예측 가능한 인터페이스
    - 버튼의 역할이 명확해야 함
    - 인터페이스 동작이 일관되어야 함
    - 예상 가능한 위치와 패턴 사용

ⓒ2026.정수아.All rights reserved.

- **안정성(Stability) : 사용자가 안심하고 사용할 수 있는 UX**
  - 예측 가능한 인터페이스



ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

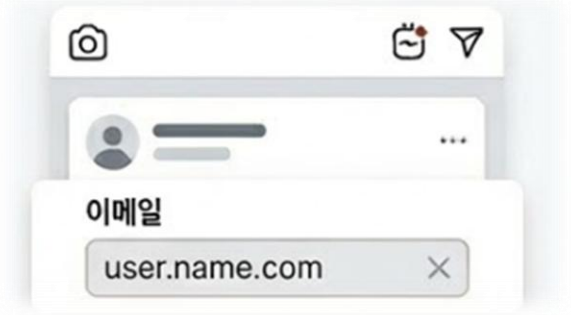
- **안정성(Stability) : 사용자가 안심하고 사용할 수 있는 UX**
  - 실수 방지 설계
    - 입력 형식 제한
    - 잘못된 입력 사전 차단
    - 중요한 행동에 확인 단계 제공

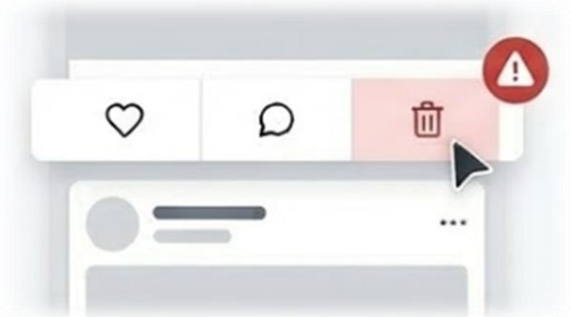
ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- **안정성(Stability) : 사용자가 안심하고 사용할 수 있는 UX**
  - 실수 방지 설계

나쁜 UX





좋은 UX

ⓒ2026.정수아.All rights

UX 핵심 원칙

- **안정성(Stability) : 사용자가 안심하고 사용할 수 있는 UX**
  - 명확한 상태 전달
    - 처리 과정 표시
    - 진행 상태 안내
    - 결과 피드백 제공

ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- **안정성(Stability) : 사용자가 안심하고 사용할 수 있는 UX**
  - 명확한 상태 전달

나쁜 UX

좋은 UX

ⓒ2026.정수아.All rights

UX 핵심 원칙

- **안정성(Stability) : 사용자가 안심하고 사용할 수 있는 UX**
  - 오류 발생 시 명확한 안내
    - 오류 원인 안내
    - 해결 방법 제시
    - 다시 시도할 수 있는 기능 제공

ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- **안정성(Stability) : 사용자가 안심하고 사용할 수 있는 UX**
  - 오류 발생 시 명확한 안내

ⓒ2026.정수아.All rights

- **안정성(Stability) : 사용자가 안심하고 사용할 수 있는 UX**

안정성 요소	핵심 의미
예측 가능성	사용자가 행동의 결과를 미리 예상할 수 있도록 인터페이스 설계
실수 방지 설계	사용자가 실수하기 어렵도록 인터페이스 구성
명확한 상태 전달	시스템의 진행 상태와 결과를 사용자에게 알려주는 인터페이스 설계
오류 대응	오류 발생 시 원인과 해결 방법을 제공

ⓒ2026.정수아.All rights reserved.

UX 핵심 원칙

- **좋은 UX를 판단 기준**

기준	의미	사용자 입장에서의 느낌
명확성	무엇을 해야 하는지 바로 이해됨	“뭘 해야 할지 알겠다”
흐름성	과정이 끊기지 않고 자연스럽게 이어짐	“막힘 없이 진행된다”
효율성	최소한의 노력으로 목표 달성	“빠르고 편하다”
안정성	시스템 상태가 예측 가능하고 신뢰 가능	“지금 뭐가 진행 중인지 안다”

ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - 사용자 목표 정의
  - 사용자 행동 흐름 설계
  - 화면 단위 분리
  - UI 구조 배치

ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - 사용자 목표 정의
    - 사용자는 왜 이 화면에 들어왔는가?
    - 무엇을 하려고 하는가?
    - 핵심 목표 1개로 압축

기능은 여러 개일 수 있지만, 목표는 하나여야 한다

ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - 사용자 행동 흐름 설계
    - 시작 → 과정 → 완료
    - 단계 최소화
    - 되돌림, 막힘 제거

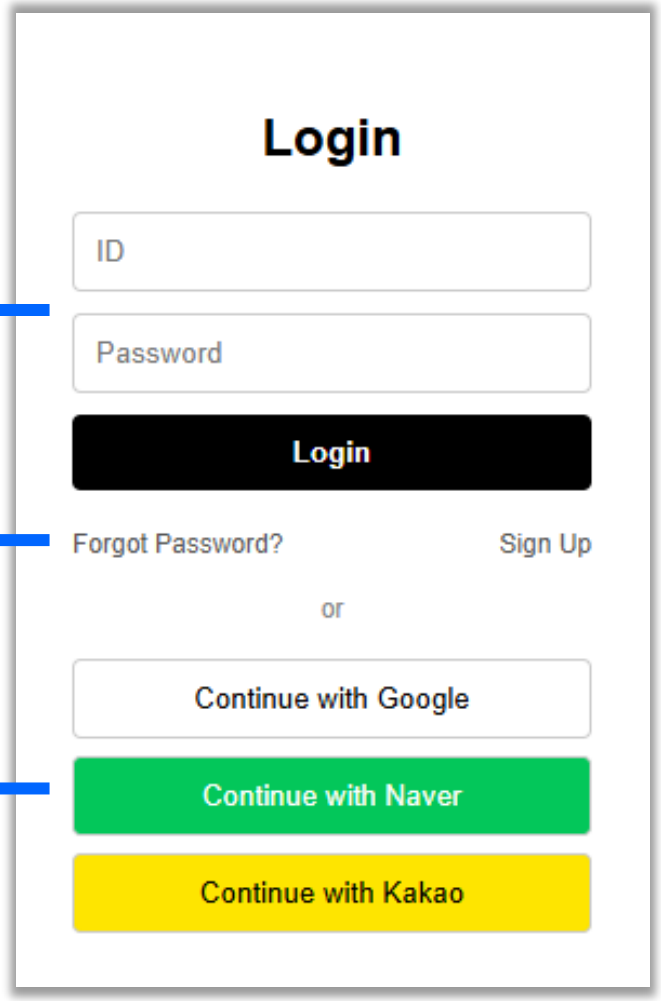
ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - 사용자 행동 흐름 설계

로그인 기능

회원가입 기능

소셜 로그인 기능



아이디 입력
비밀번호 입력
로그인

ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - 화면 단위 분리
    - 원칙 : 한 화면 = 하나의 목적
    - 한 화면에서 사용자가 수행할 행동이나 달성할 목표를 명확하게 정의

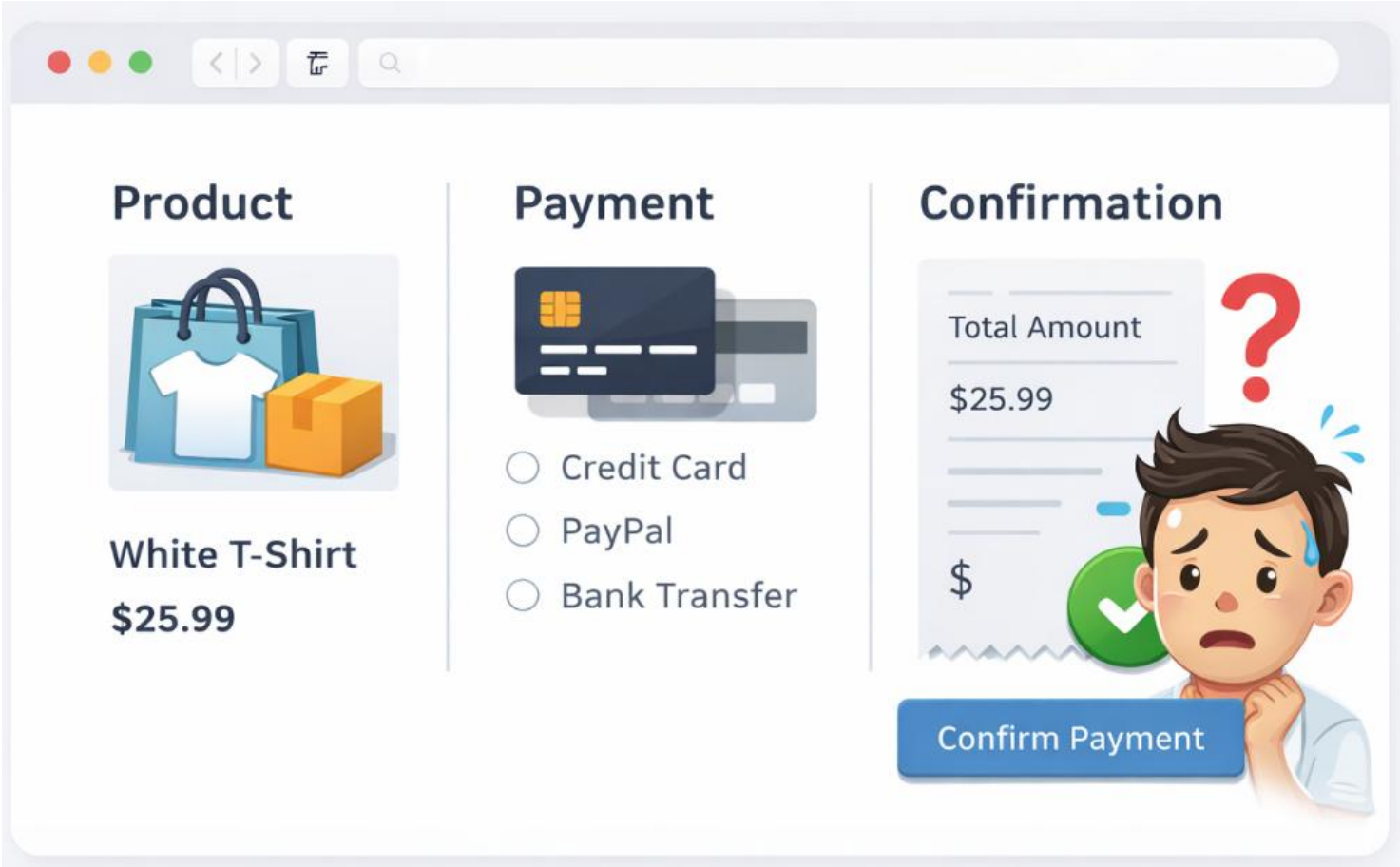
ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - 화면 단위 분리
    - 사용자의 행동 흐름은 그대로 구현되지 않는다.
    - 어디서 끊고, 어떻게 나눌 것인가?

구분	잘못된 분할	적절한 분할
흐름	흐름 단절	자연스러운 단계 전환
정보량	정보 과다	필요한 정보만 제시
인지 부담	인지 부담 증가	인지 부담 감소
사용자 이해	목표 파악 어려움	명확한 목표 인식
결과	오류 및 이탈 발생	안정적인 사용 흐름

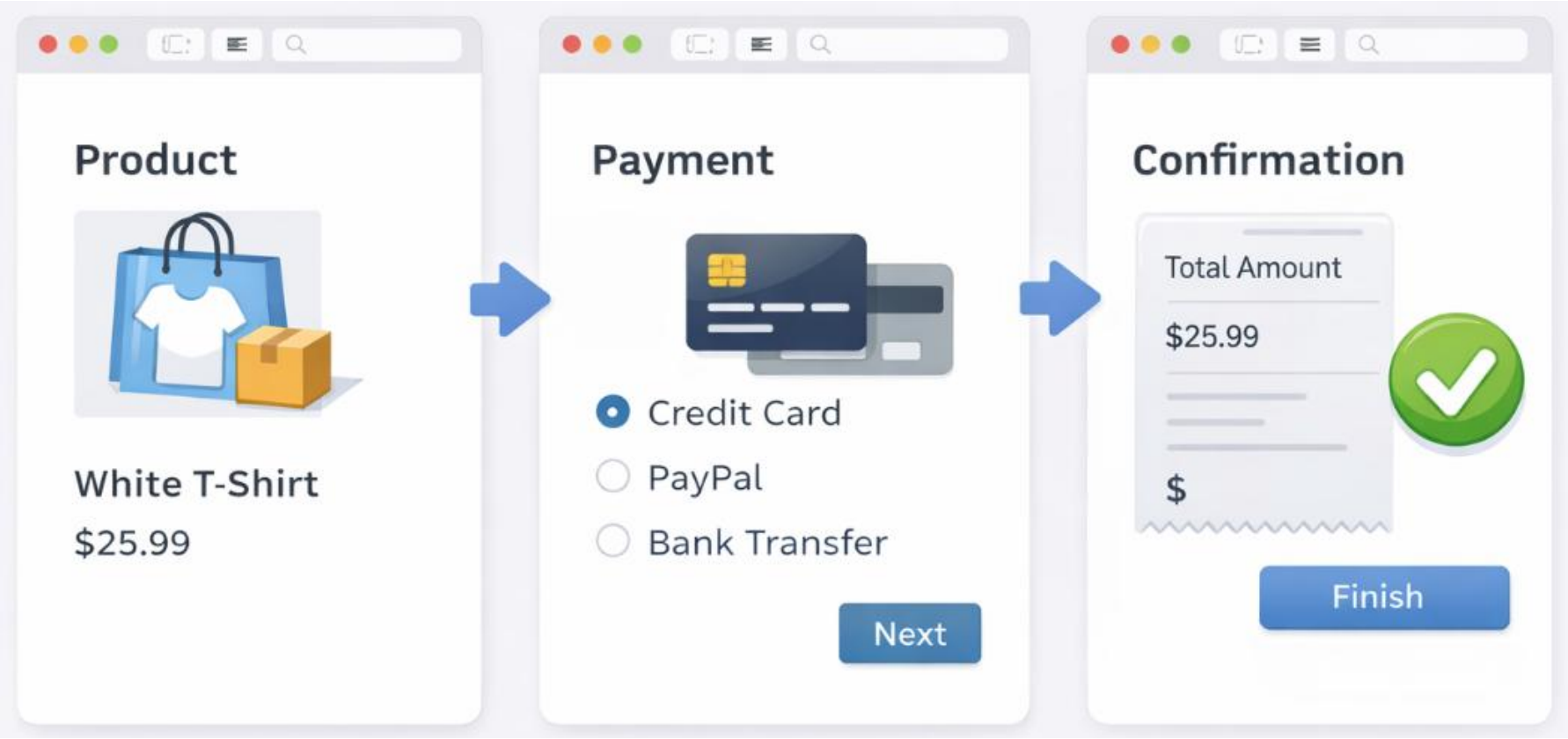
ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - 화면 단위 분리



ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - 화면 단위 분리



ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - UI 구조 배치
    - 정보 우선 순위
    - 시선 흐름
    - 행동 유도 요소
    - 피드백 제공

ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - UI 구조 배치 : 정보 우선 순위
    - 우선 순위 판단 기준

기준	예시
화면을 즉시 이해할 수 있는가?	페이지 제목, 단계 표시
사용자의 현재 상태를 알 수 있는가?	선택된 상품 요약, 진행 상태
사용자가 어디서부터 시작해야 하는가?	첫 입력 필드, 주요 선택 요소
초기 판단에 필요한 핵심 정보인가?	상품명, 핵심 요약 정보

ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - UI 구조 배치 : 시선 흐름
    - 사용자는 화면을 무작위로 보지 않는다.
    - 일정한 패턴을 따라 시선을 이동한다.

구분	설명	예시
F형 패턴	텍스트를 훑어보듯 읽는 패턴	기사, 상세 페이지, 설정 화면
Z형 패턴	화면을 빠르게 스캔하는 패턴	랜딩 페이지, 프로모션 화면

ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - UI 구조 배치 : 시선 흐름

F형 패턴

Z형 패턴

ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - UI 구조 배치 : 행동 유도 요소(CTA, Call To Action)
    - 명확성
    - 가시성
    - 위치
    - 집중

ⓒ2026.정수아.All rights reserved.

- **UI 구조 설계 흐름**
  - UI 구조 배치 : 피드백 제공
    - 피드백의 역할
      - 상태 전달(진행 중, 완료, 오류)
      - 행동 확인(정상 처리 여부)
      - 다음 행동 유도
    - 피드백 유형
      - 진행 상태 : 로딩, 프로그래스 바
      - 완료 안내 : 완료 메시지, 체크 표시
      - 오류 안내 : 에러 메시지, 입력 경고

ⓒ2026.정수아.All rights reserved.

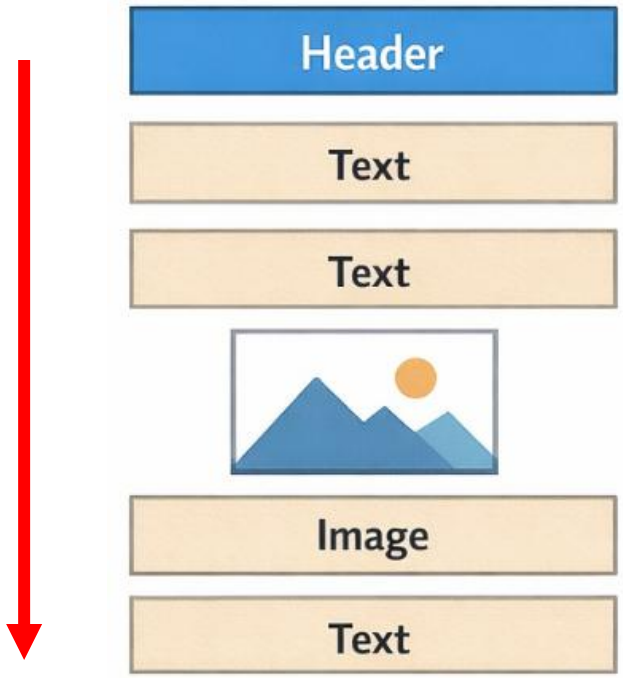
UI 구현 관점에서 UX 정리

- **UX 기준과 연결**
  - 명확성
    - 목표, 구조
  - 흐름성
    - 사용자 흐름
  - 효율성
    - 단계 최소화
  - 안정성
    - 피드백 설계

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **웹의 시작은 '문서'였다**
  - 레이아웃 개념이 없었음



초기의 웹

VS

현재의 웹

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **레이아웃을 위한 우회적 구현**
  - position
  - float
  - table

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **position이란?**
  - 요소의 위치를 결정하는 CSS 속성
  - 기준 + 이동으로 구성
    - 기준 : static, relative, absolute, fixed, sticky
    - 위치 : top, right, bottom, left
  - 흐름을 유지하거나 흐름에서 벗어날 수 있음

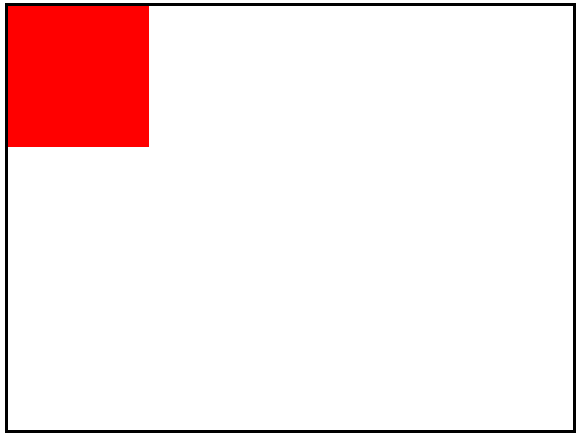
ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **position**
  - static
    - 기본값

main.html

```
<div class="card">
  <div class="box"></div>
</div>
```



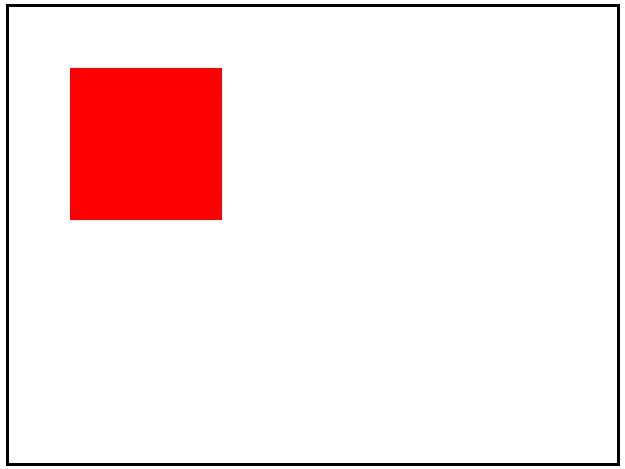
style.css

```
.card {
  width: 200px;
  height: 150px;
  border: 1px solid black;
}

.box {
  width: 50px;
  height: 50px;
  background: red;
  position: static;
  top: 20px;
  left: 20px;
}
```

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **position**
  - relative
    - 자기 자신을 기준으로 이동
    - 원래 자리 그대로 유지됨



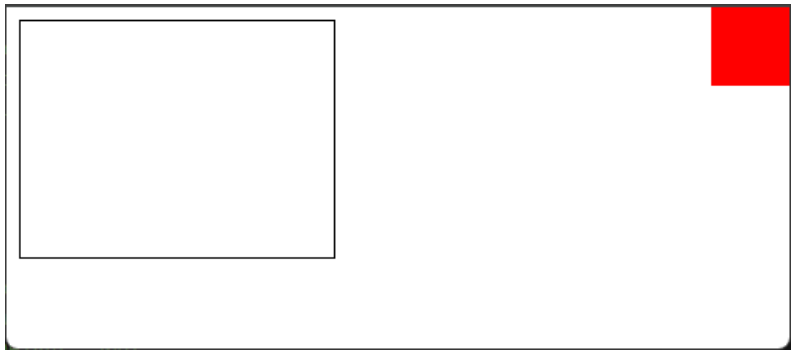
style.css

```
.card {
  width: 200px;
  height: 150px;
  border: 1px solid black;
}

.box {
  width: 50px;
  height: 50px;
  background: red;
  position: relative;
  top: 20px;
  left: 20px;
}
```

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **position**
  - absolute
    - 가장 가까운 부모를 기준으로 이동
    - 위치를 자유롭게 지정 가능



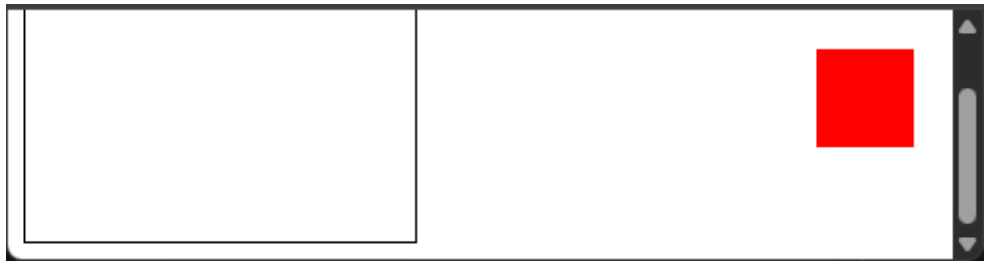
style.css

```
.card {
  width: 200px;
  height: 150px;
  border: 1px solid black;
}

.box {
  width: 50px;
  height: 50px;
  background: red;
  position: absolute;
  top: 0;
  right: 0;
}
```

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **position**
  - fixed
    - 화면 기준으로 위치를 고정
    - 스크롤해도 따라옴



style.css

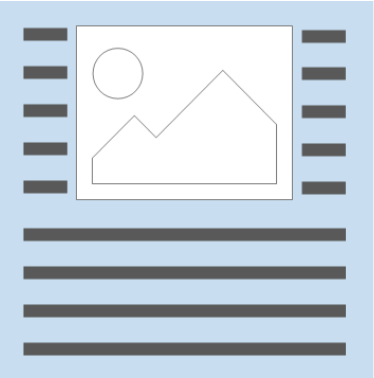
```
.card {
  width: 200px;
  height: 150px;
  border: 1px solid black;
}

.box {
  width: 50px;
  height: 50px;
  background: red;
  position: fixed;
  top: 20px;
  right: 20px;
}
```

ⓒ2026.정수아.All rights reserved.

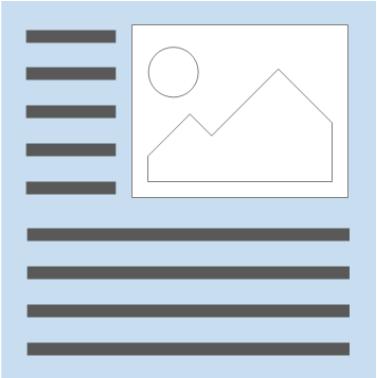
레이아웃의 시작 : 과거의 방법들

- **float란?**
  - 요소를 좌우로 띄우는 속성
  - 목적 : 텍스트 흐름 제어
    - 이미지 옆에 텍스트 배치
    - 자연스러운 흐름 유지
  - 레이아웃 도구가 아님

float : left



float : center



float : right

- **float의 문제**
  - 부모가 자식의 높이를 인식하지 못함
  - 요소가 예상과 다르게 배치

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

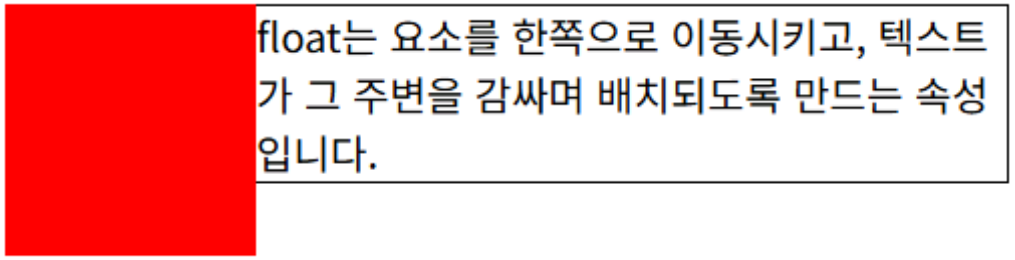
- **float**
  - left

main.html

```
<div class="box">></div>
<p class="text">
  float는 요소를 한쪽으로 이동시키고, 텍스트가
  그 주변을 감싸며 배치되도록 만드는 속성입니다.
</p>
```

style.css

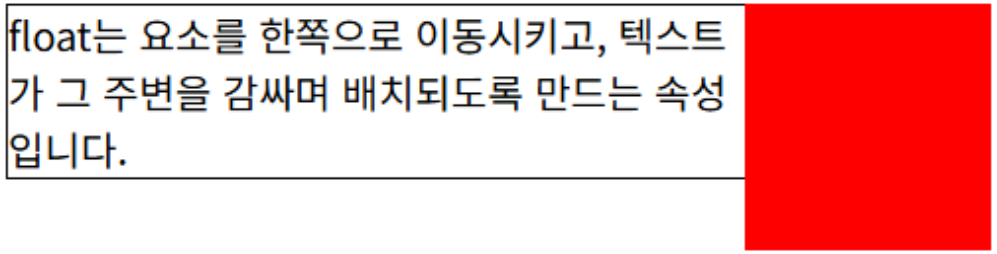
```
.box {
  width: 100px;
  height: 100px;
  background: red;
  float: left;
}

.text {
  border: 1px solid black;
}
```



ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **float**
  - right



style.css

```
.box {
  width: 100px;
  height: 100px;
  background: red;
  float: right;
}

.text {
  border: 1px solid black;
}
```

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **float**
  - none

float는 요소를 한쪽으로 이동시키고, 텍스트가 그 주변을 감싸며 배치되도록 만드는 속성입니다.

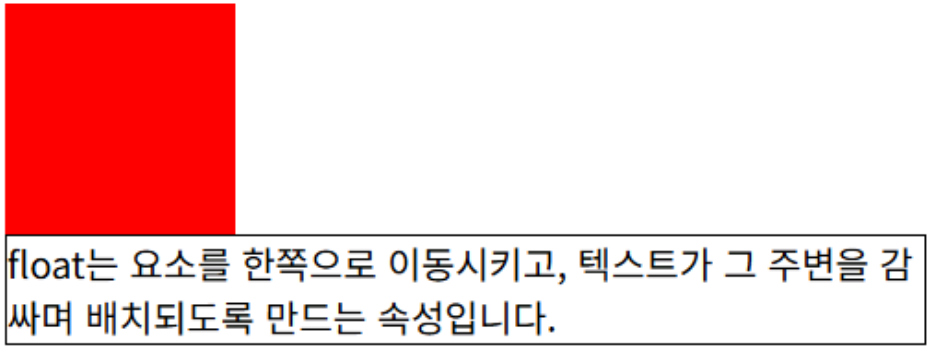
style.css

```
.box {
  width: 100px;
  height: 100px;
  background: red;
  float: none;
}

.text {
  border: 1px solid black;
}
```

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **float**
  - clear:left



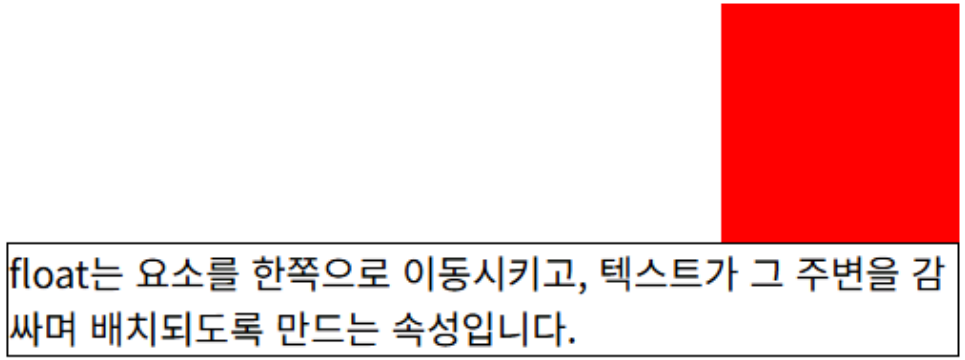
style.css

```
.box {
  width: 100px;
  height: 100px;
  background: red;
  float: left;
}

.text {
  border: 1px solid black;
  clear: left;
}
```

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **float**
  - clear:right



style.css

```
.box {
  width: 100px;
  height: 100px;
  background: red;
  float: right;
}

.text {
  border: 1px solid black;
  clear: right;
}
```

ⓒ2026.정수아.All rights reserved.

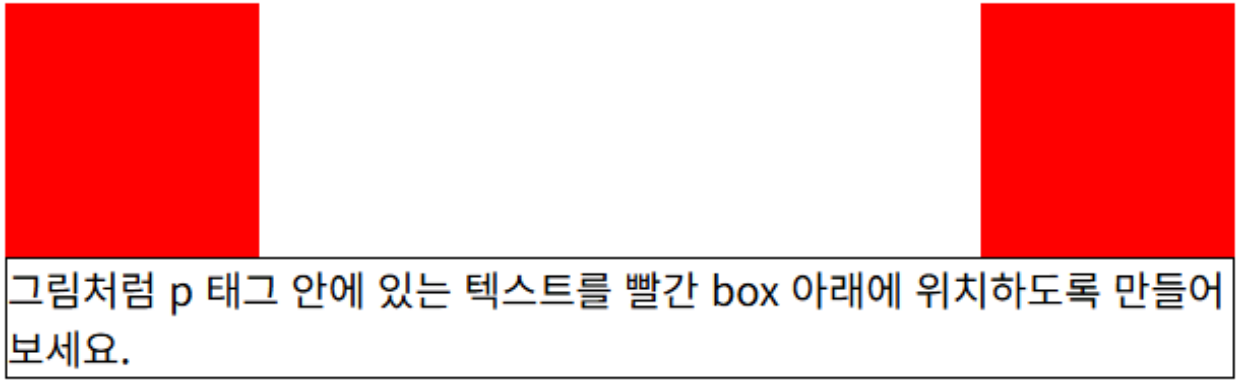
레이아웃의 시작 : 과거의 방법들

- **float**

main.html

```
<div class="box left"></div>
<div class="box right"></div>

<p class="text">
   그림처럼 p 태그 안에 있는 텍스트를
   빨간 box 아래에 위치하도록 만들어보세요.
<p>
```

실행 결과



ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **table이란?**
  - 데이터를 표현하는 태그
  - 행(row)과 열(column) 구조
  - 정렬이 쉬움
  - 목적
    - 표 형태로 데이터 표현
    - 일정한 구조 유지

ⓒ2026.정수아.All rights reserved.

레이아웃의 시작 : 과거의 방법들

- **position, float, table 공통 문제점**
  - 레이아웃 전용 도구가 아님
  - position
    - 흐름이 깨진다.
  - float
    - 예외가 계속 발생한다.
  - table
    - 데이터 구조로 화면을 구성한다.

ⓒ2026.정수아.All rights reserved.

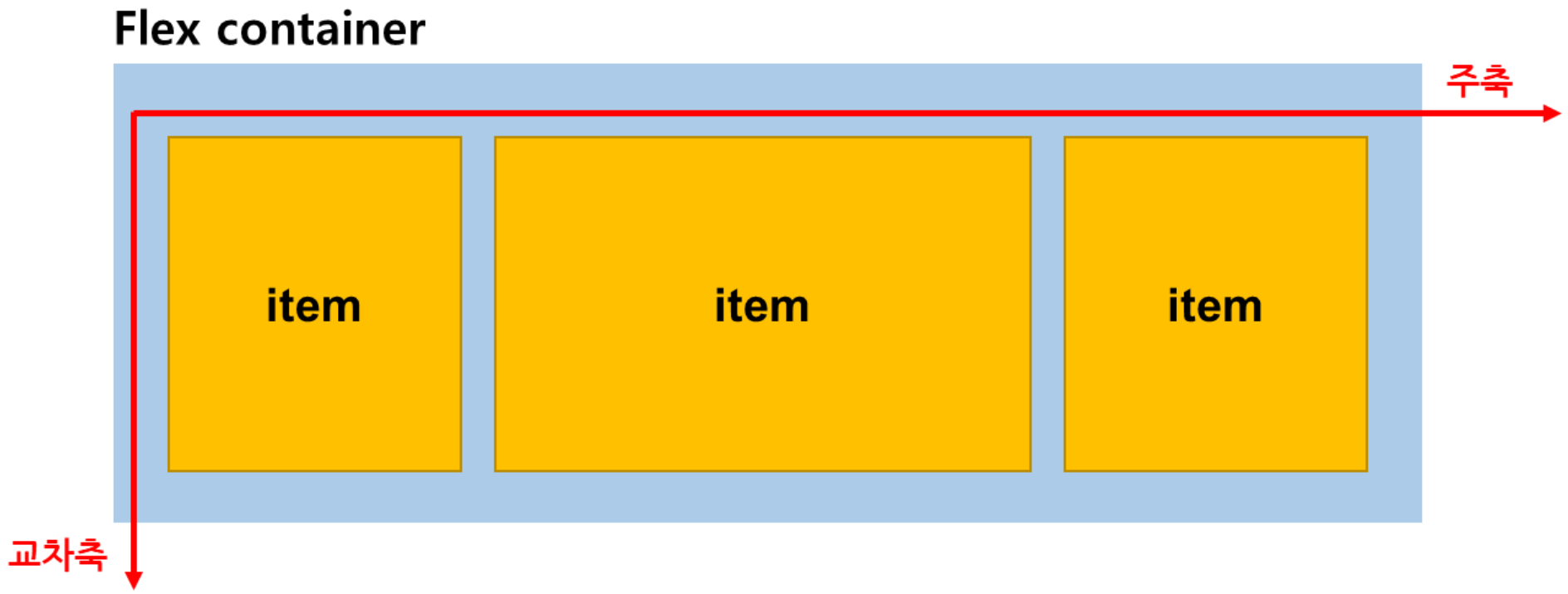
Flexbox 개념과 역할

- **Flexbox**
  - Flexible Box Layout
  - 행(Row) 또는 열(Column)을 기준으로 요소를 정렬하고 배치하는 1차원 레이아웃 방식
  - 특징
    - 흐름을 유지하며 요소를 배치
    - 공간을 자동으로 분배
    - 구조가 단순하고 일관적

ⓒ2026.정수아.All rights reserved.

Flexbox 개념과 역할

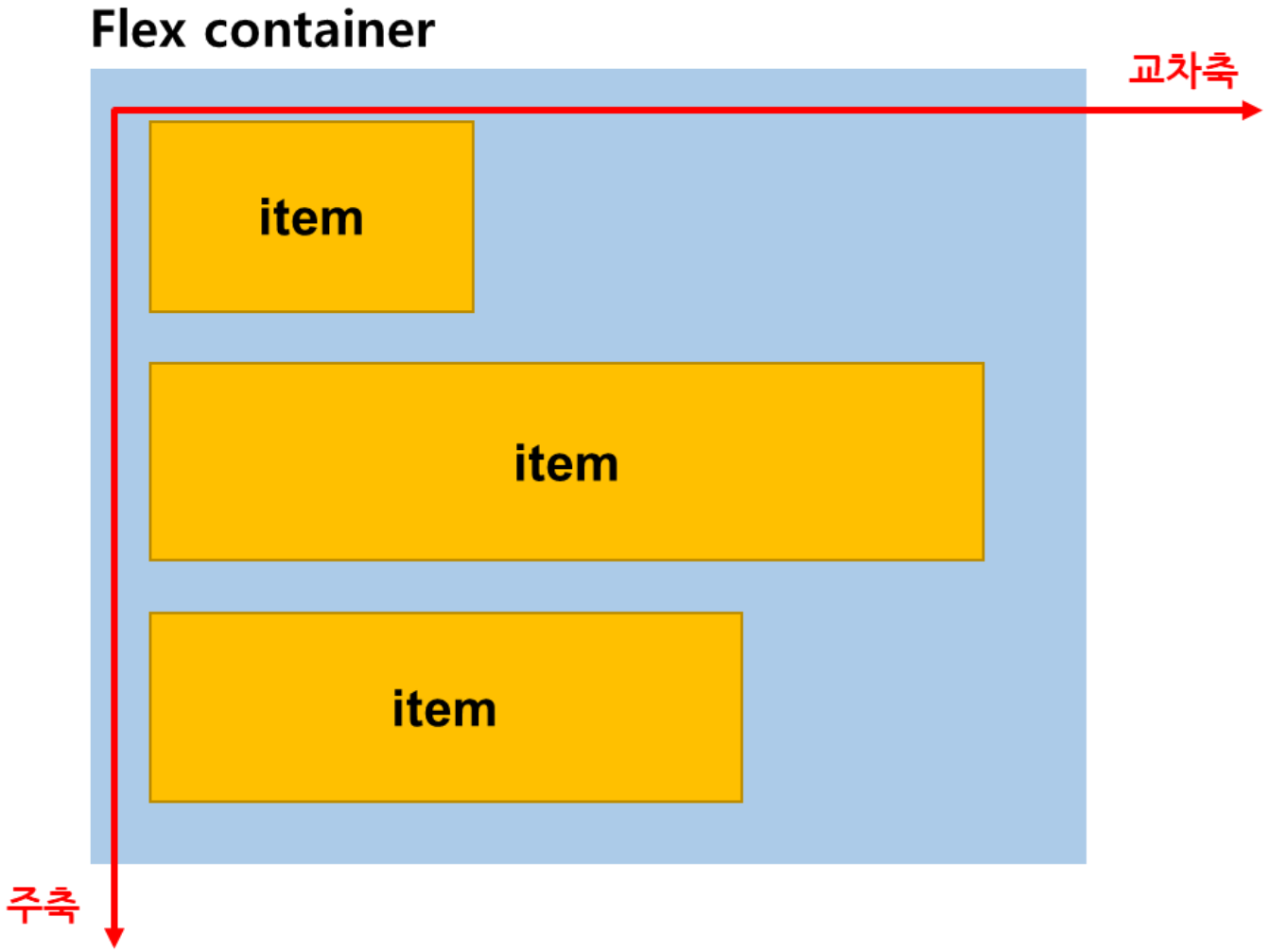
- **Flexbox 구조**



ⓒ2026.정수아.All rights reserved.

Flexbox 개념과 역할

- **Flexbox 구조**



ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **Flex container 속성**
  - display:flex
    - Flexbox 시작
    - 부모 요소를 Flex container로 만들어줌
  - flex-direction
    - 방향 설정(주축 결정)
    - 값 : row, column

ⓒ2026.정수아.All rights reserved.

- **flex-direction**

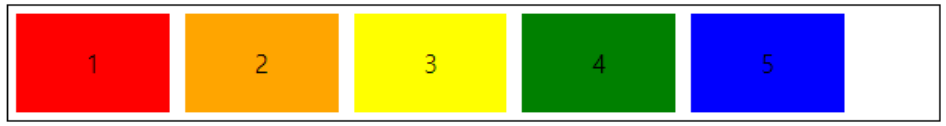
main.html

```
<div class="container">
  <div class="item item1"><p>1</p></div>
  <div class="item item2"><p>2</p></div>
  <div class="item item3"><p>3</p></div>
  <div class="item item4"><p>4</p></div>
  <div class="item item5"><p>5</p></div>
</div>
```

style.css

```
.container {
    border: 1px solid black;
    display: flex;

}

.item {
    padding: 5px 45px;
    margin: 5px;
}

.item1 { background : red; }
.item2 { background : orange; }
.item3 { background : yellow; }
.item4 { background : green; }
.item5 { background : blue; }
```

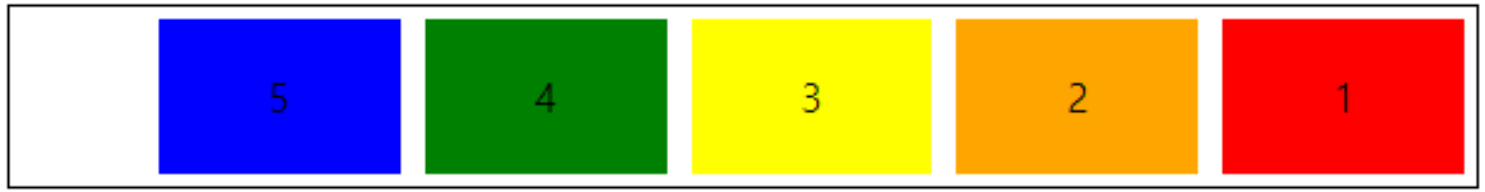
실행 결과



ⓒ2026.정수아.All rights reserved.

- **flex-direction**
  - flex-direction : row

  - flex-direction : row-reverse



ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

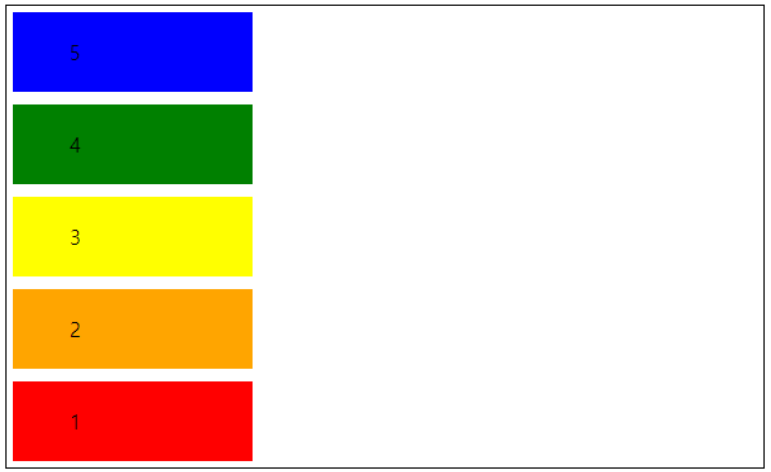
- **flex-direction**
  - flex-direction : column

  - .item { width:100px; }

ⓒ2026.정수아.All rights reserved.

- **flex-direction**
  - flex-direction : column-reverse

  - .item { width:100px; }



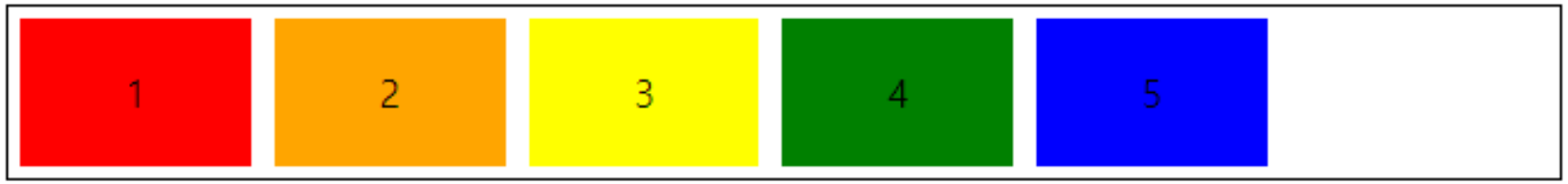
ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **Flex container 속성**
  - flex-wrap
    - 한 줄 유지 또는 여러 줄로 나누는 속성
    - 값 : nowrap(기본값), wrap, wrap-reverse

ⓒ2026.정수아.All rights reserved.

- **flex-wrap**
  - flex-wrap : nowrap



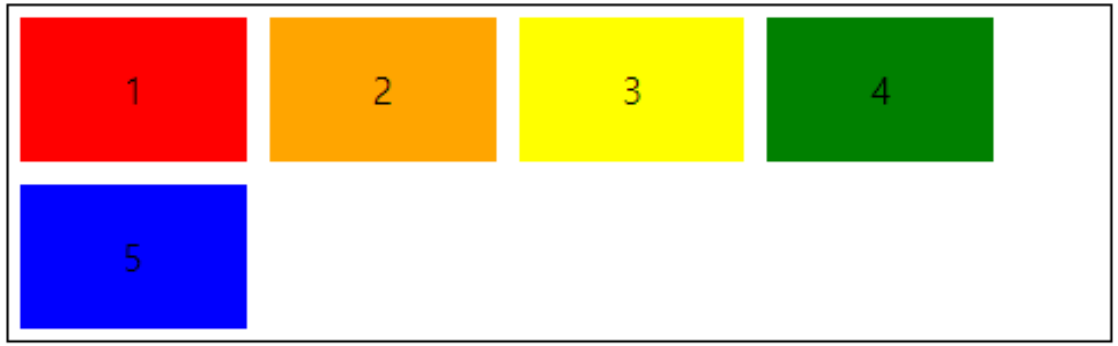
화면을 늘렸을 때

화면을 줄였을 때

ⓒ2026.정수아.All rights reserved.

- **flex-wrap**
  - flex-wrap : wrap

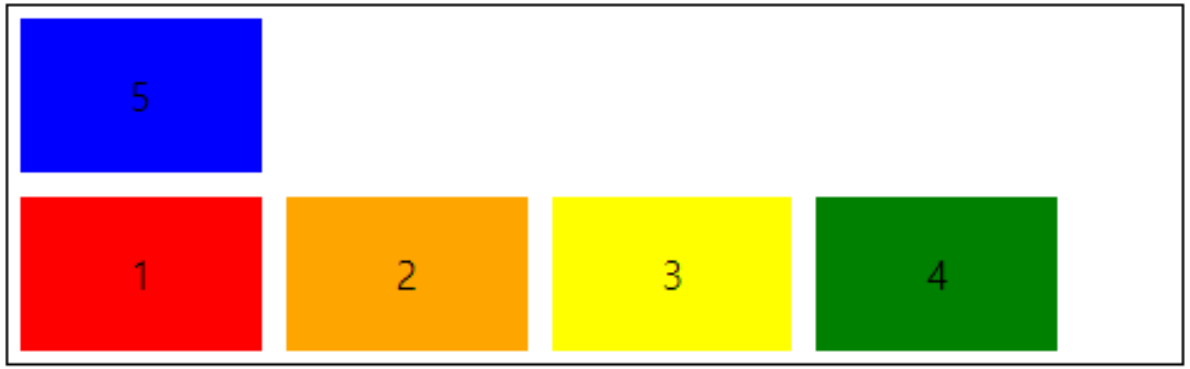
화면을 늘렸을 때



화면을 줄였을 때

ⓒ2026.정수아.All rights reserved.

- **flex-wrap**
  - flex-wrap : wrap-reverse



ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **Flex container 속성**
  - justify-content
    - 주축 방향으로 요소를 정렬
    - 값
      - flex-start
      - center
      - flex-end
      - space-between
      - space-around
      - space-evenly

ⓒ2026.정수아.All rights reserved.

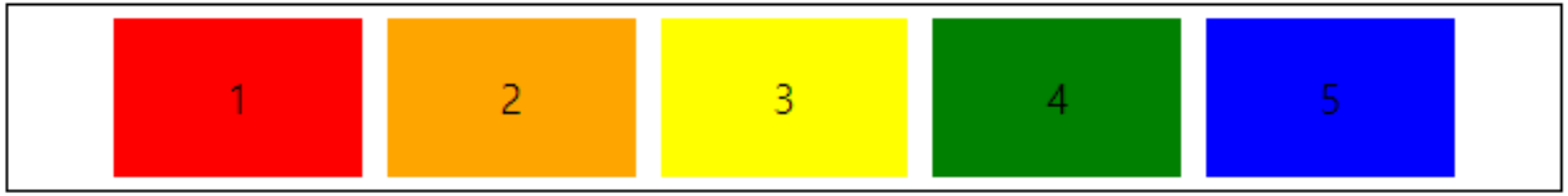
- **justify-content**
  - justify-content:flex-start

  - justify-content:flex-end

ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **justify-content**
  - justify-content:center

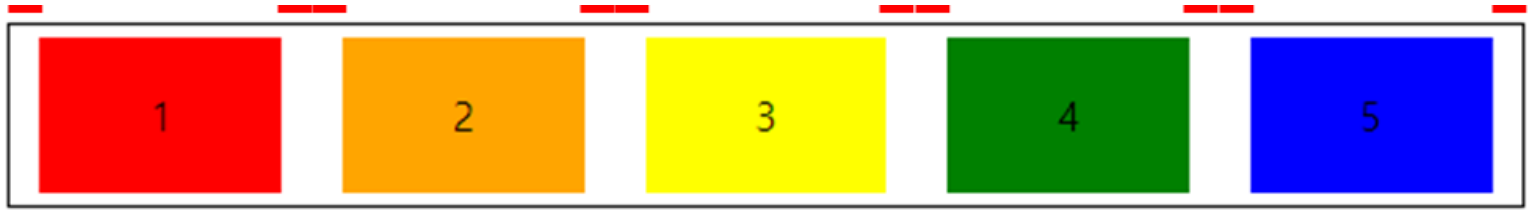


  - justify-content:space-between

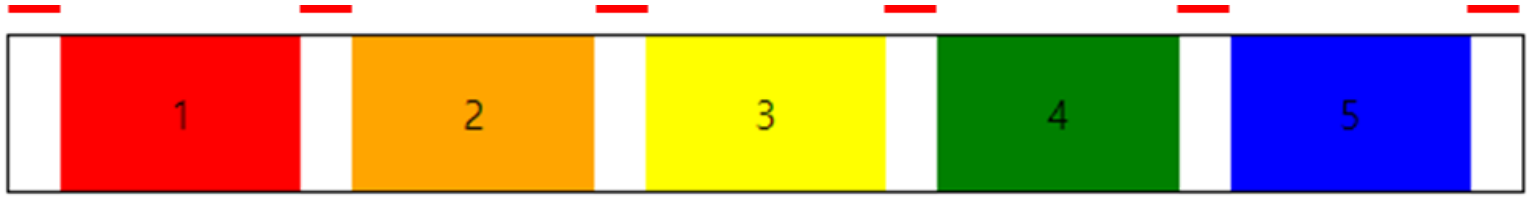
ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **justify-content**
  - justify-content:space-around



  - justify-content:space-evenly



ⓒ2026.정수아.All rights reserved.

- **실무에서 많이 사용하는 justify-content 패턴**

  - 가운데 정렬

```
.container {
        display: flex;
        justify-content: center;
}
```

  - 오른쪽 정렬

```
.container {
        display: flex;
        justify-content: flex-end;
}
```

  - 양쪽 끝 정렬

```
.container {
        display: flex;
        justify-content: space-between;
}
```

ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **Flex container 속성**
  - align-items
    - 교차축 방향으로 요소를 정렬
    - 값
      - flex-start
      - flex-end
      - center
      - baseline
      - stretch

ⓒ2026.정수아.All rights reserved.

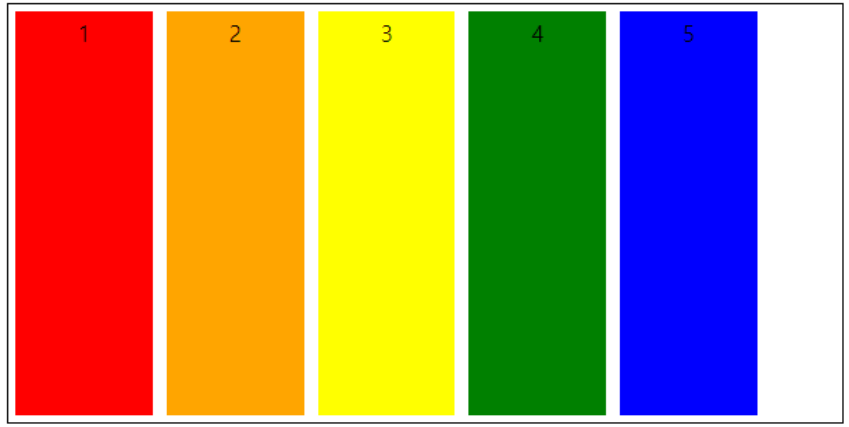
Flexbox 기반 UI 레이아웃 구현

- **align-items**

main.html

```
<div class="container">
  <div class="item item1"><p>1</p></div>
  <div class="item item2"><p>2</p></div>
  <div class="item item3"><p>3</p></div>
  <div class="item item4"><p>4</p></div>
  <div class="item item5"><p>5</p></div>
</div>
```

실행 결과



style.css

```
.container {
    border: 1px solid black;
    display: flex;
    width : 600px;
    height : 300px;
    flex-direction : row;
}

.item {
    padding: 5px 45px;
    margin: 5px;
}

.item1 { background : red; }
.item2 { background : orange; }
.item3 { background : yellow; }
.item4 { background : green; }
.item5 { background : blue; }
```

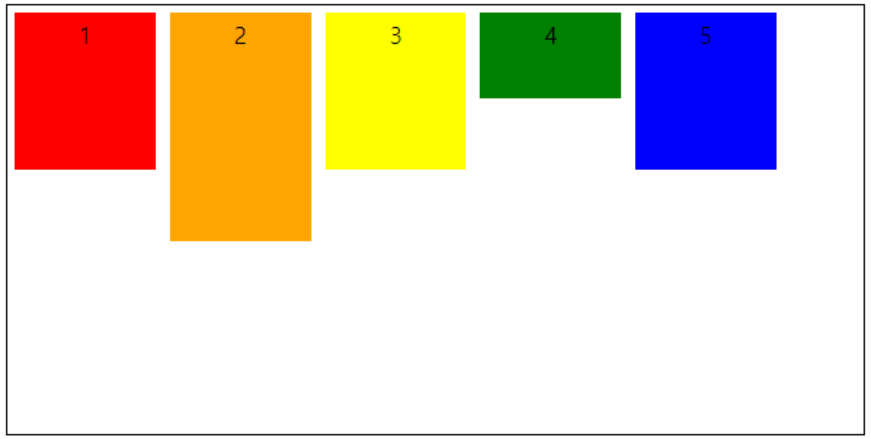
ⓒ2026.정수

- **align-items**

main.html

```
<div class="container">
  <div class="item item1"><p>1</p></div>
  <div class="item item2"><p>2</p></div>
  <div class="item item3"><p>3</p></div>
  <div class="item item4"><p>4</p></div>
  <div class="item item5"><p>5</p></div>
</div>
```

실행 결과



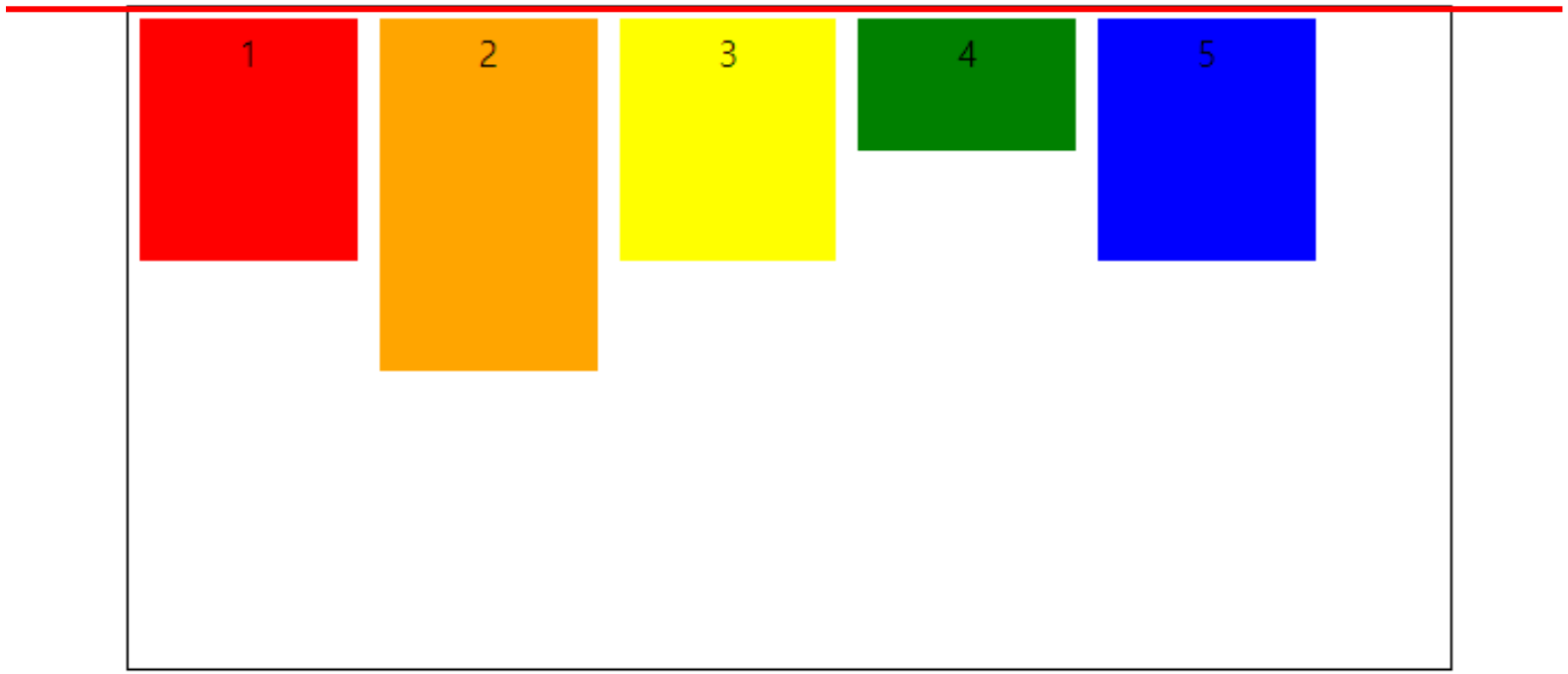
style.css

```
.container {
    border: 1px solid black;
    width : 600px;
    height : 300px;
    display: flex;
    flex-direction : row;
}

.item {
    padding: 5px 45px;
    margin: 5px;
}

.item1 { background : red; height : 100px;}
.item2 { background : orange; height : 150px;}
.item3 { background : yellow; height : 100px;}
.item4 { background : green; height : 50px;}
.item5 { background : blue; height : 100px;}
```

ⓒ2026.정수

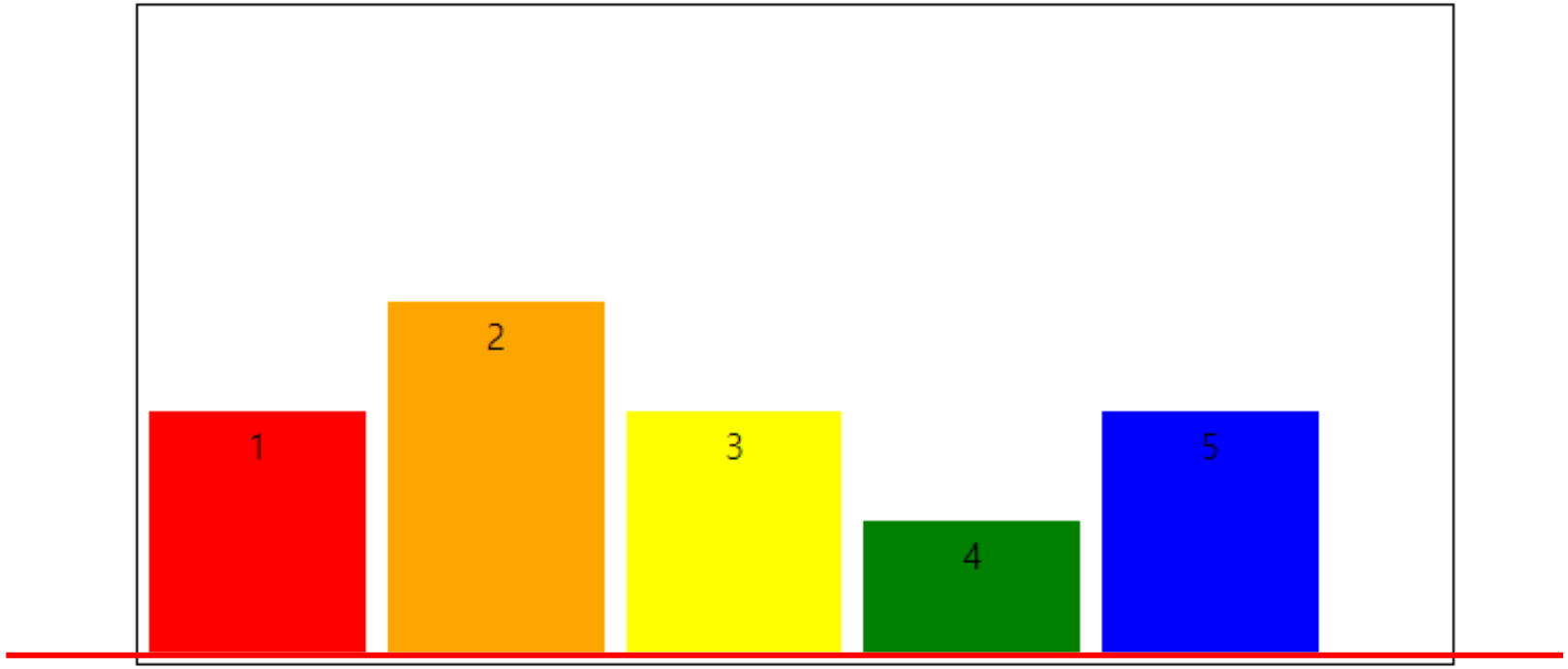
- **align-items**
  - align-items:flex-start

교차축의 시작점에 배치



ⓒ2026.정수아.All rights reserved.

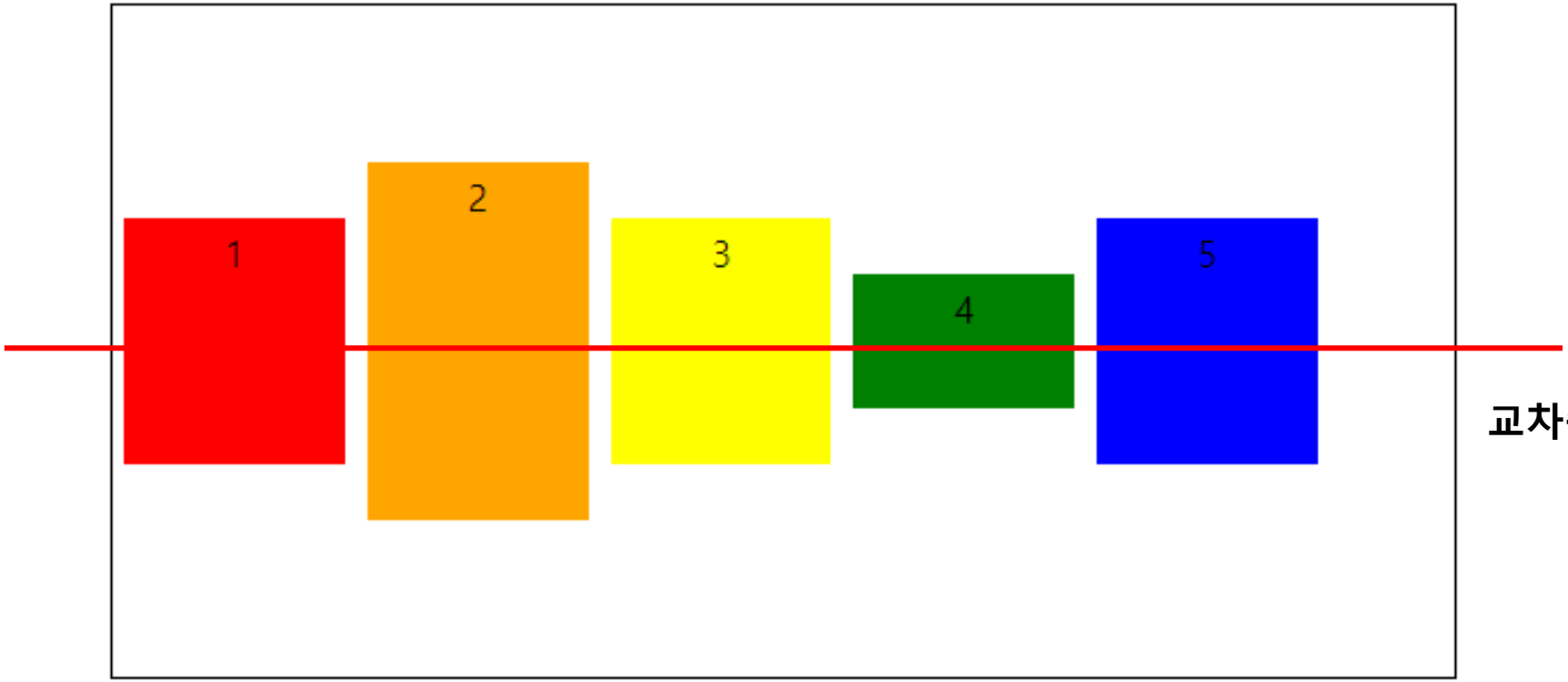
- **align-items**
  - align-items:flex-end



교차축의 끝점에 배치

ⓒ2026.정수아.All rights reserved.

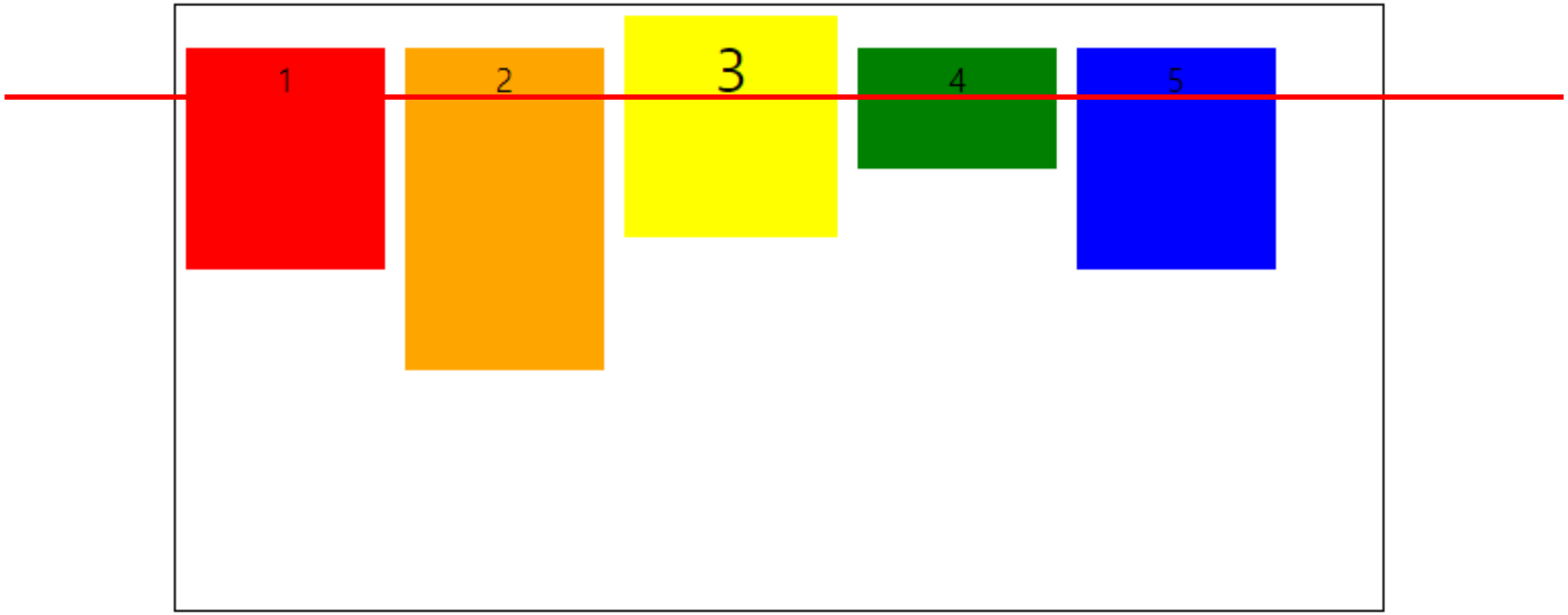
- **align-items**
  - align-items:center



교차축의 중앙에 배치

ⓒ2026.정수아.All rights reserved.

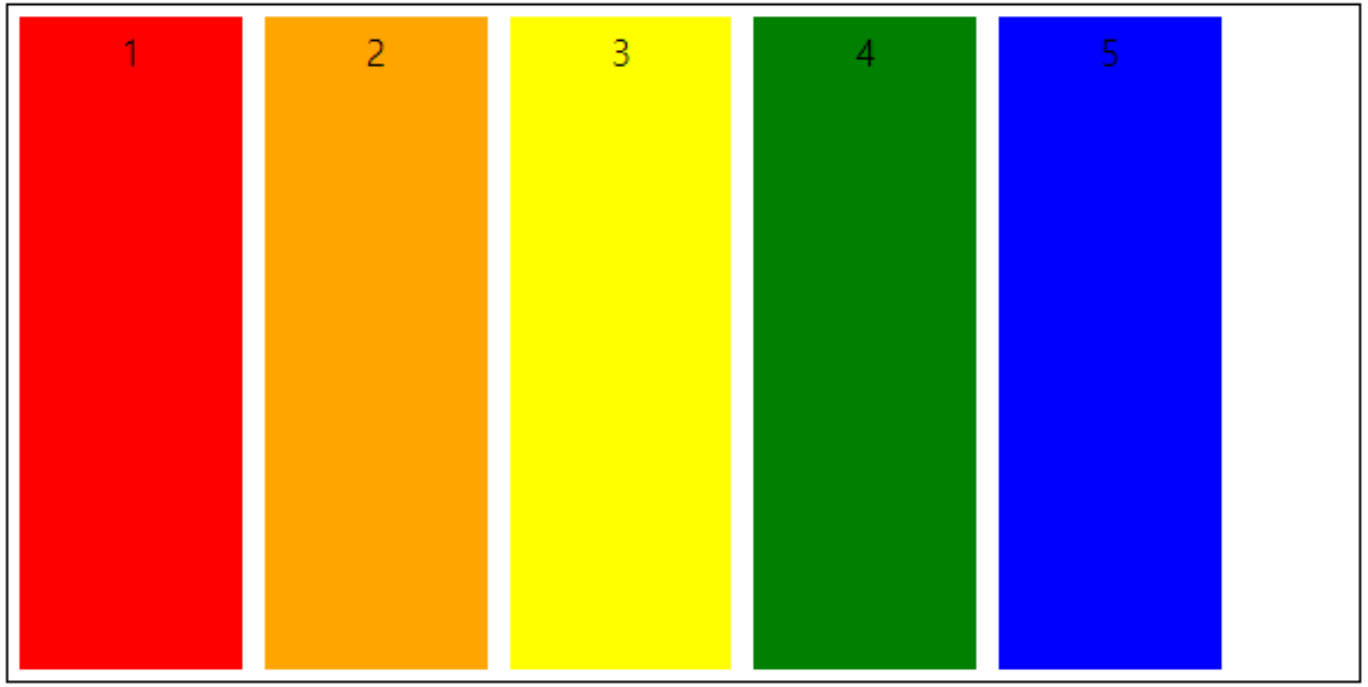
- **align-items**
  - align-items:baseline



.item3의 글자 크기 30px

ⓒ2026.정수아.All rights reserved.

- **align-items**
  - align-items:stretch



ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **Flex container 속성**

속성	역할	기준 축
flex-direction	방향 설정	주축 결정
flex-wrap	줄바꿈 설정	주축
justify-content	정렬	주축
align-items	정렬	교차축

ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **Flex item**
  - Flex container 안에 있는 자식 요소들

ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **Flex item 속성**
  - align-self
    - Flex item을 각각 원하는 위치로 배치할 수 있음
    - 값
      - flex-start
      - flex-end
      - center
      - baseline
      - stretch

ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **align-self**

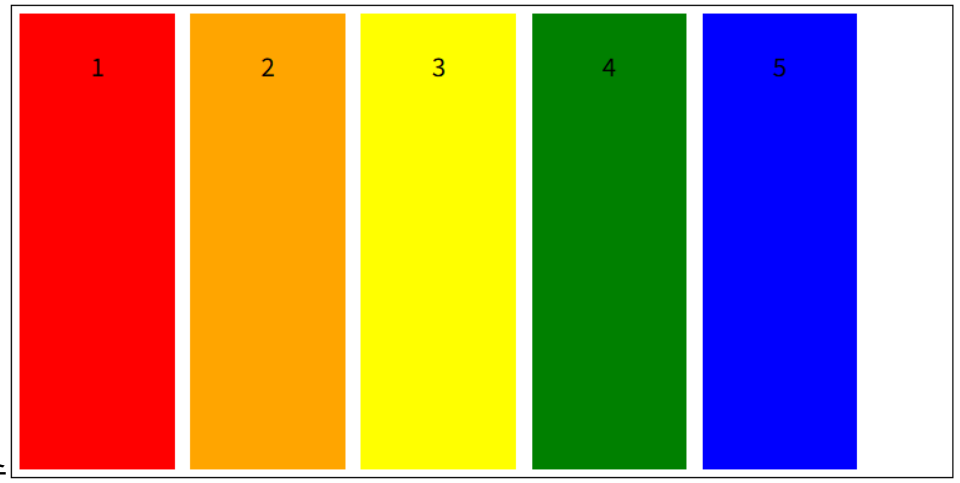
main.html

```
<div class="container">
  <div class="item item1"><p>1</p></div>
  <div class="item item2"><p>2</p></div>
  <div class="item item3"><p>3</p></div>
  <div class="item item4"><p>4</p></div>
  <div class="item item5"><p>5</p></div>
</div>
```

style.css

```
.container {
  border: 1px solid black;
  width: 600px;
  height: 300px;
  display: flex;
  flex-direction: row;
}


.item {
  padding: 5px 45px;
  margin: 5px;
}

.item1 { background: red; }
.item2 { background: orange; }
.item3 { background: yellow; }
.item4 { background: green; }
.item5 { background: blue; }
```

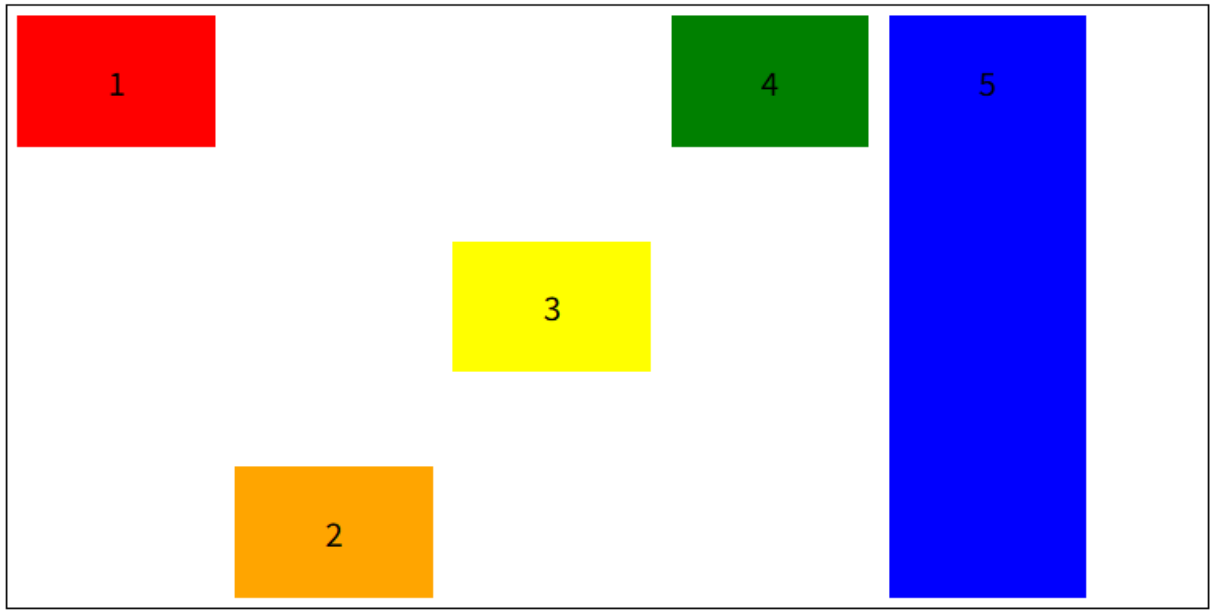
실행 결과



ⓒ2026.정수

- **align-self**

```
.item1 { align-self: flex-start; }
.item2 { align-self: flex-end; }
.item3 { align-self: center; }
.item4 { align-self: baseline; }
.item5 { align-self: stretch; }
```

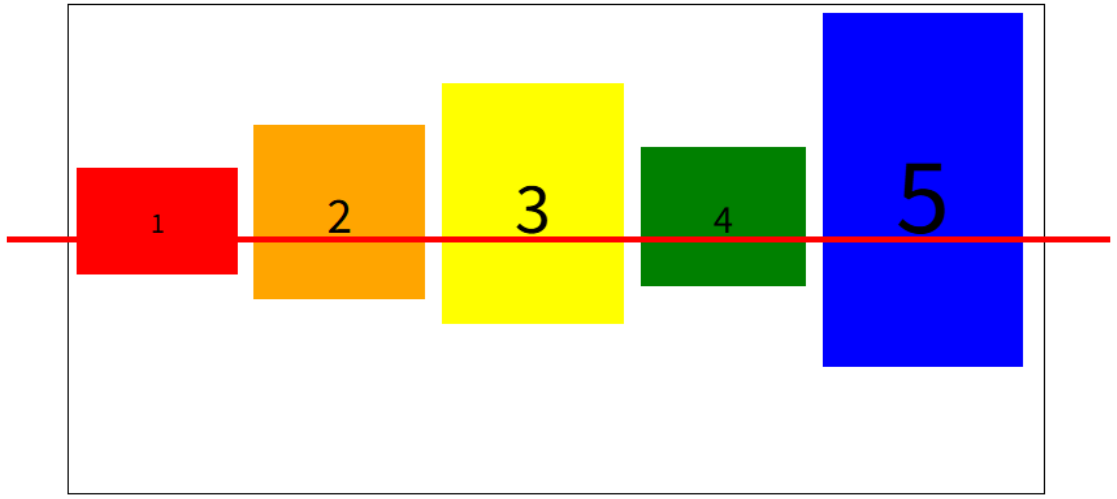


ⓒ2026.정수아.All rights reserved.

- **align-self**
  - baseline 적용

```
/* 각 item의 font-size를 다르게 적용 */
.item1 p {
  font-size: 16px;
}
.item2 p {
  font-size: 28px;
}
.item3 p {
  font-size: 40px;
}
.item4 p {
  font-size: 22px;
}
.item5 p {
  font-size: 60px;
}
```

```
/* 각 item에 baseline 적용 */
.item1,
.item2,
.item3,
.item4,
.item5 {
  align-self: baseline;
}
```



ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **Flex item 속성**
  - flex
    - Flex item의 크기와 비율을 결정하는 속성
    - 값
      - flex-grow
      - flex-basis
      - flex-shrink

ⓒ2026.정수아.All rights reserved.

- **flex-grow 속성**

main.html

```
<div class="container">
   <div class="item item1"><p>1<p></div>
   <div class="item item2"><p>2</p></div>
   <div class="item item3"><p>3<p></div>
</div>
```

style.css

```
.container {
        border: 1px solid black;
        display: flex;
}

.item {
        padding: 5px 45px;
        margin: 5px;
}
```

```
.item1 {
        background: red;
}

.item2 {
        background: orange;
}

.item3 {
        background: yellow;
}
```

ⓒ2026.정수아.All rights reserved.

- **flex-grow 속성**
  - flex-grow
    - 남는 공간을 비율로 나눔
    - 기본값 : 0
      - 숫자가 클수록 더 많이 차지
  - .item1 { flex-grow : 1 }

ⓒ2026.정수아.All rights reserved.

- **flex-grow 속성**
  - .item2 { flex-grow : 1 }

  - .item3 { flex-grow : 1 }

ⓒ2026.정수아.All rights reserved.

- **flex-basis 속성**
  - flex-basis
    - flex 요소의 초기 크기(기준 크기)를 설정
    - 모든 계산의 출발점

main.html

```
<div class="container">
  <div class="item item1">1</div>
  <div class="item item2">2</div>
</div>
```

style.css

```
.container {
  width: 500px;
  border: 1px solid black;
  background: yellow;
  display: flex;
}

.item {
  border: 1px solid black;
}

.item1 {
  background: red;
  flex-basis: 100px;
}

.item2 {
  background: orange;
  flex-basis: 100px;
}
```

ⓒ2026.정수아.All rights reserved.

- **flex-basis 속성**
  - flex-grow 적용

style.css

```
.item1 {
  background: red;
  flex-basis: 100px;
  flex-grow: 1;
}

.item2 {
  background: orange;
  flex-basis: 100px;
  flex-grow: 1;
}
```

ⓒ2026.정수아.All rights reserved.

- **flex-basis 속성**
  - flex-grow 적용

style.css

```
.item1 {
  background: red;
  flex-basis: 200px;
  flex-grow: 1;
}

.item2 {
  background: orange;
  flex-basis: 100px;
  flex-grow: 1;
}
```

ⓒ2026.정수아.All rights reserved.

- **flex-basis 속성**
  - flex-grow 적용

style.css

```
.item1 {
  background: red;
  flex-basis: 0;
  flex-grow: 1;
}

.item2 {
  background: orange;
  flex-basis: 0;
  flex-grow: 1;
}
```

ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

- **실무에서 많이 사용하는 flex:1**
  - 모든 요소가 동일한 크기로 나뉜다.
  - flex: 1
    - grow: 1
    - shrink: 1
    - basis: 0

main.html

```
<div class="container">
   <div class="item item1">1</div>
   <div class="item item2">2</div>
</div>
```

style.css

```
.container {
  width: 500px;
  border: 1px solid black;
  background: yellow;
  display: flex;
}

.item {
  border: 1px solid black;
  flex: 1;
}

.item1 {
  background: red;
}

.item2 {
  background: orange;
}
```

ⓒ2026.정수아.All rights reserved.

- **width vs flex-basis**

style.css

```
.container {
  width: 500px;
  border: 1px solid black;
  background: yellow;
  display: flex;
}

.item {
  border: 1px solid black;
  flex: 1;                /* basis: 0 */
  width: 100px;
}
.item1 {
  background: red;
}

.item2 {
  background: orange;
}
```

ⓒ2026.정수아.All rights reserved.

- **flex-shrink 속성**
  - 부족한 공간을 줄임
  - 기본값 : 1

main.html

```
<div class="container">
   <div class="item item1">1</div>
   <div class="item item2">2</div>
</div>
```

style.css

```
.container {
  width: 500px;
  border: 1px solid black;
  background: yellow;
  display: flex;
}

.item {
  border: 1px solid black;
  width: 400px;
}

.item1 {
  background: red;
}

.item2 {
  background: orange;
}
```

ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

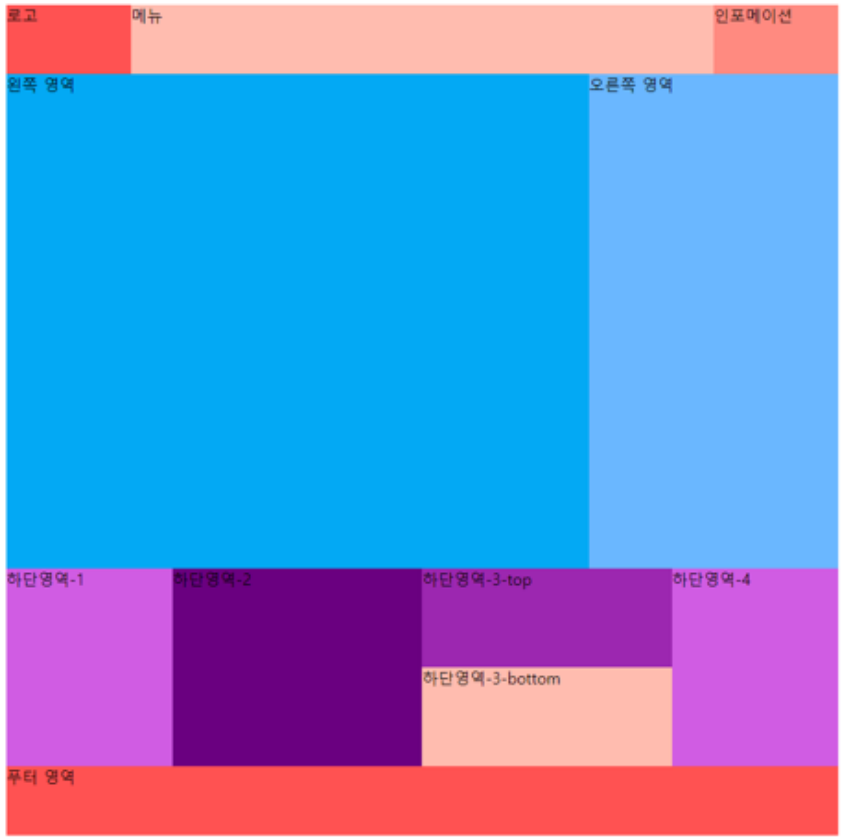
- **Flexbox froggy**
  - https://flexboxfroggy.com/#ko

- **Flexbox Defense**
  - http://www.flexboxdefense.com/

ⓒ2026.정수아.All rights reserved.

Flexbox 기반 UI 레이아웃 구현

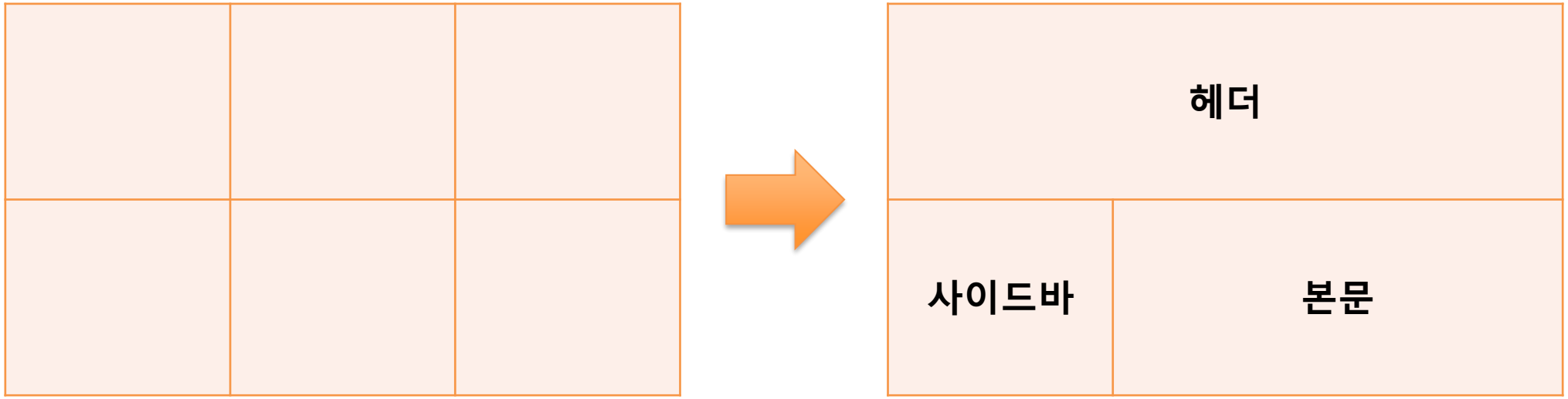
- **Flexbox의 한계**
  - Flexbox는 1차원 레이아웃
    - 한 번에 한 방향만 중심적으로 다룸
      - row 기준 정렬
      - column 기준 정렬
    - 전체 화면 구조를 설계하기에는 한계가 있음



ⓒ2026.정수아.All rights reserved.

Grid 개념과 구조

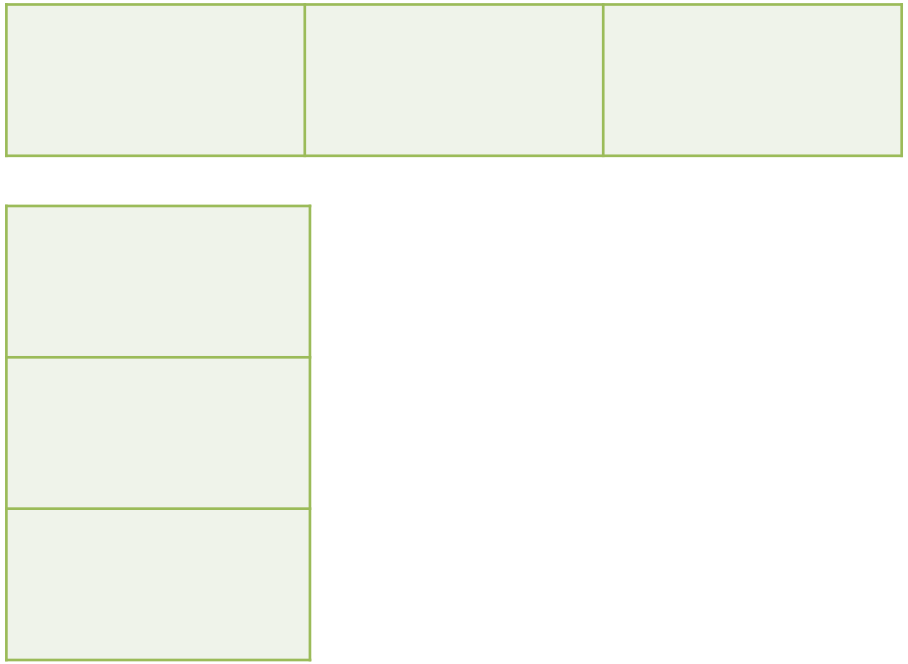
- **Grid**
  - 2차원 레이아웃 시스템
  - 행(row)과 열(column)을 동시에 설계 가능
  - 전체 화면 구조를 직관적으로 나눌 수 있음
  - 복잡한 레이아웃을 더 적은 중첩으로 설계 가능



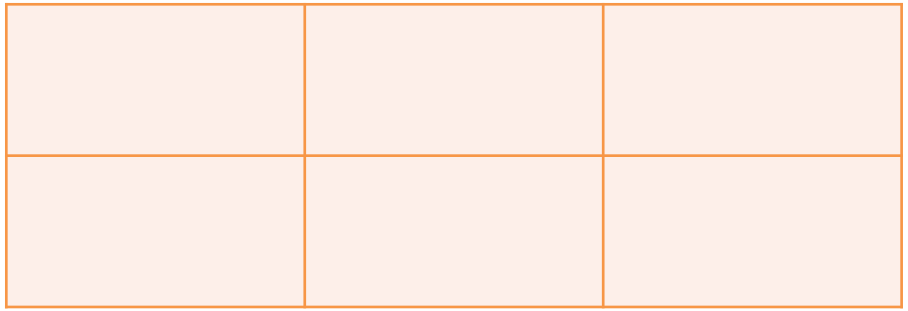
ⓒ2026.정수아.All rights reserved.

Grid 개념과 구조

- **Grid**
  - Grid와 Flexbox의 차이



Flexbox

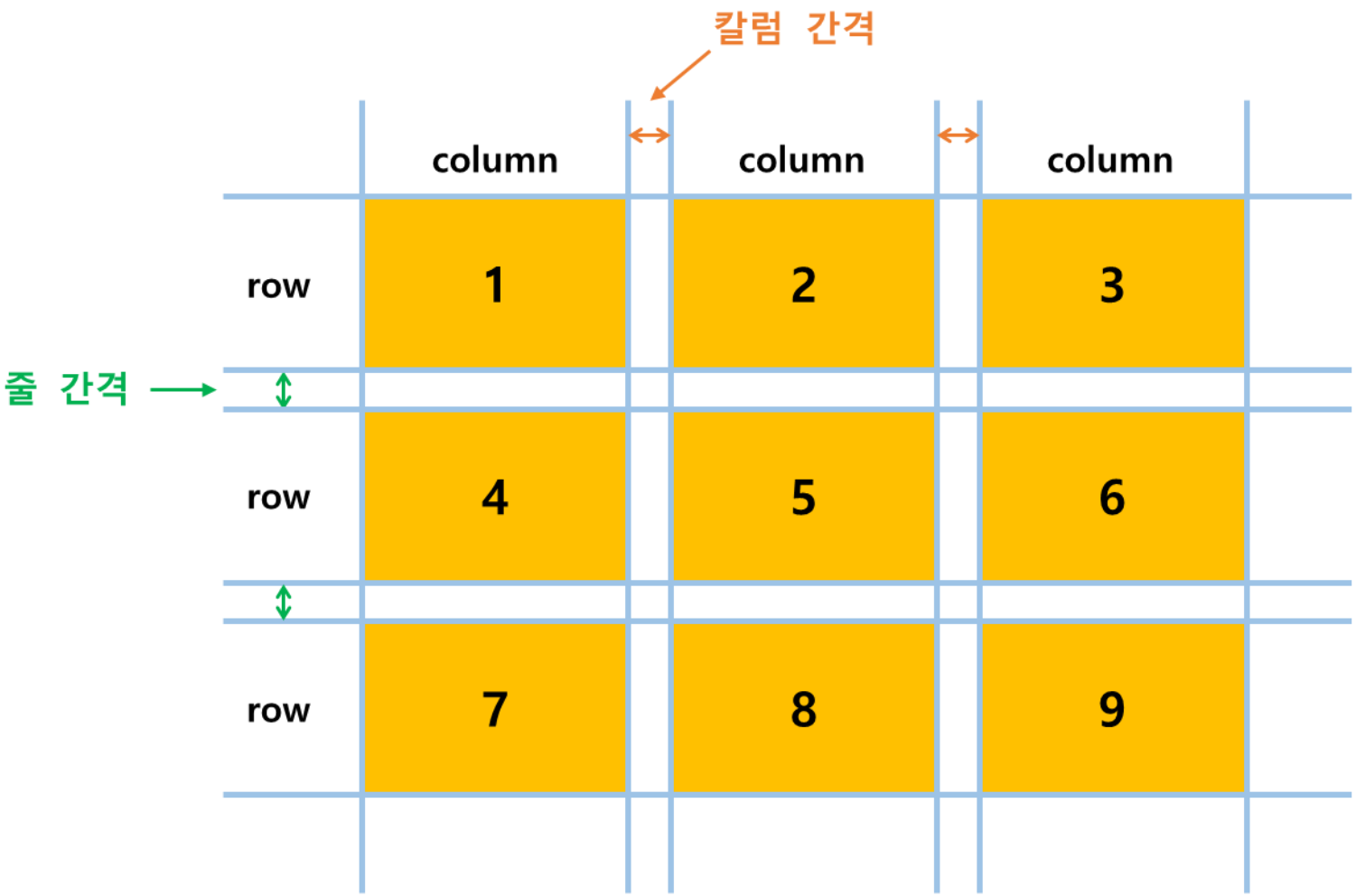


Grid

ⓒ2026.정수아.All rights reserved.

Grid 개념과 구조

- **Grid**



ⓒ2026.정수아.All rights reserved.

Grid 개념과 구조

- **Grid**
  - Grid Container
    - display: grid가 적용된 부모 요소
    - Grid 구조를 만드는 기준이 되는 영역
  - Grid Item
    - Grid Container 안에 들어 있는 자식 요소
    - 격자 안에 배치되는 실제 요소

ⓒ2026.정수아.All rights reserved.

Grid 기반 페이지 레이아웃 구현

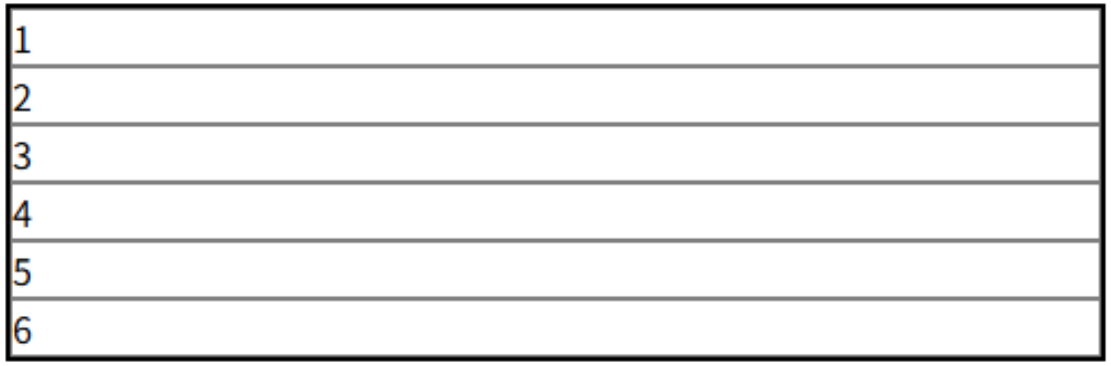
- **Grid 실습**

main.html

```
<div class="container">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
</div>
```

style.css

```
.container {
    border: 2px solid black;
    display: grid;
}

.container > div {
    border: 1px solid gray;
}
```



ⓒ2026.정수아.All rights reserved.

- **Grid 실습**
  - column 생성

style.css

```
.container {
    border: 2px solid black;
    display: grid;
    grid-template-columns: 100px 100px 100px;
}

.container > div {
    border: 1px solid gray;
}
```

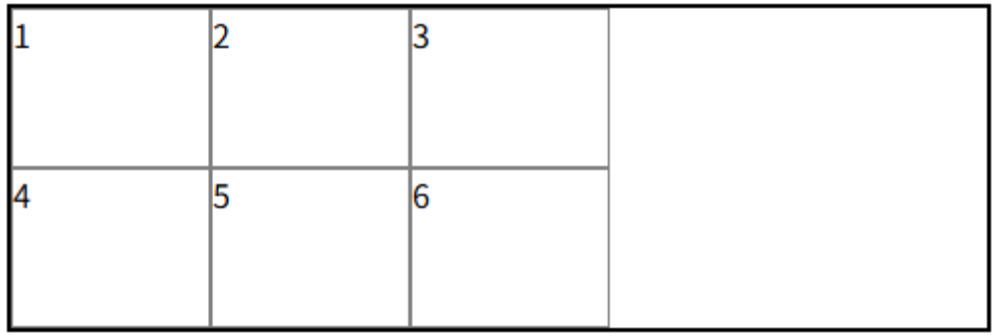
1	2	3	
4	5	6	

ⓒ2026.정수아.All rights reserved.

- **Grid 실습**
  - row 크기 지정

style.css

```
.container {
    border: 2px solid black;
    display: grid;
    grid-template-columns: 100px 100px 100px;
    grid-template-rows: 80px 80px;
}

.container > div {
    border: 1px solid gray;
}
```

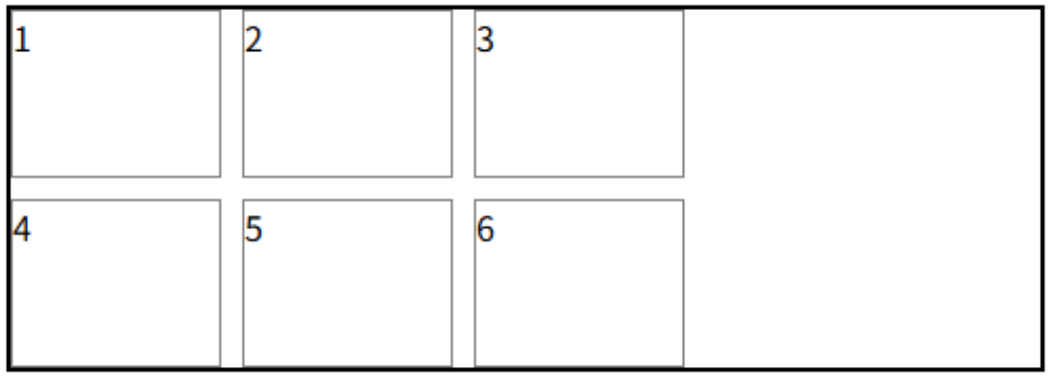


ⓒ2026.정수아.All rights reserved.

- **Grid 실습**
  - 간격 지정

style.css

```
.container {
    border: 2px solid black;
    display: grid;
    grid-template-columns: 100px 100px 100px;
    grid-template-rows: 80px 80px;
    gap: 10px;
}

.container > div {
    border: 1px solid gray;
}
```



ⓒ2026.정수아.All rights reserved.

Grid 기반 페이지 레이아웃 구현

- **Grid 실습**
  - 비율 기반 레이아웃
    - fr(fraction) 단위
      - 남은 공간을 비율로 나누는 단위

style.css

```
.container {
    border: 2px solid black;
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
    grid-template-rows: 80px 80px;
}

.container > div {
    border: 1px solid gray;
}
```

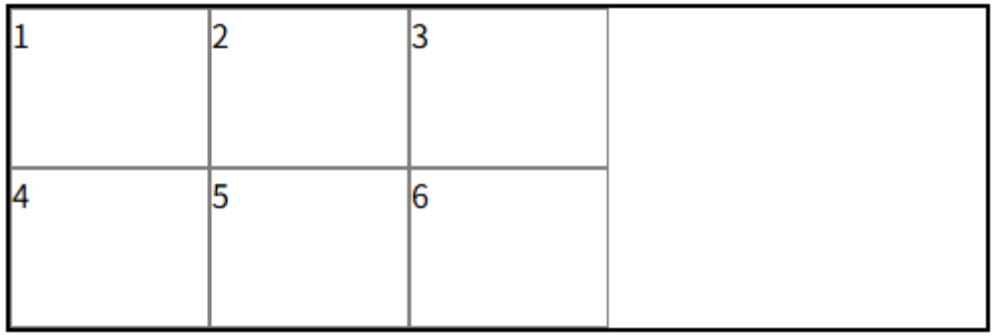
1	2	3
4	5	6

ⓒ2026.정수아.All rights reserved.

- **Grid 실습**
  - 반복 레이아웃
    - repeat(반복횟수, 값)
      - 동일한 구조를 반복할 때 사용

style.css

```
.container {
    border: 2px solid black;
    display: grid;
    grid-template-columns: repeat(3, 100px);
    grid-template-rows: 80px 80px;
}

.container > div {
    border: 1px solid gray;
}
```

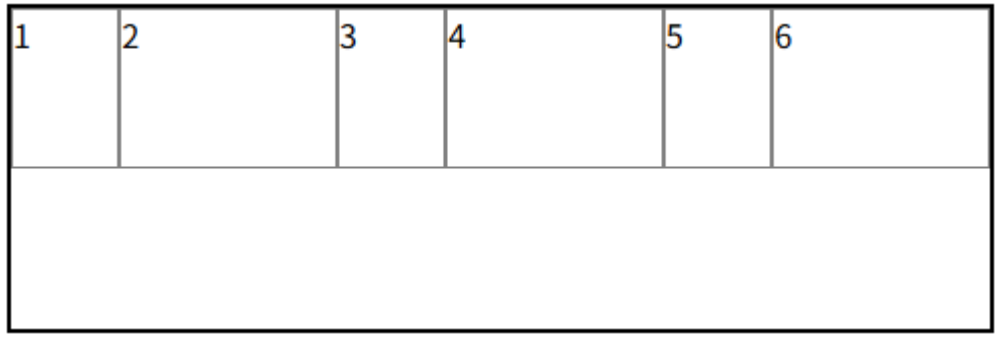


ⓒ2026.정수아.All rights reserved.

- **Grid 실습**
  - 반복 레이아웃
    - repeat(반복횟수, 값)
      - 동일한 구조를 반복할 때 사용

style.css

```
.container {
    border: 2px solid black;
    display: grid;
    grid-template-columns: repeat(3, 1fr 2fr);
    grid-template-rows: 80px 80px;
}

.container > div {
    border: 1px solid gray;
}
```

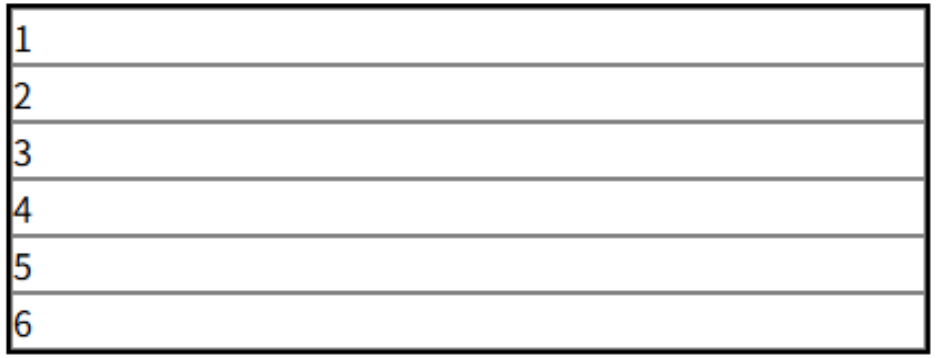


ⓒ2026.정수아.All rights reserved.

- **Grid 실습**
  - minmax(최소값, 최대값)
    - 최소값과 최대값의 범위를 정해서 아이템의 크기를 유연하게 만들어 줌

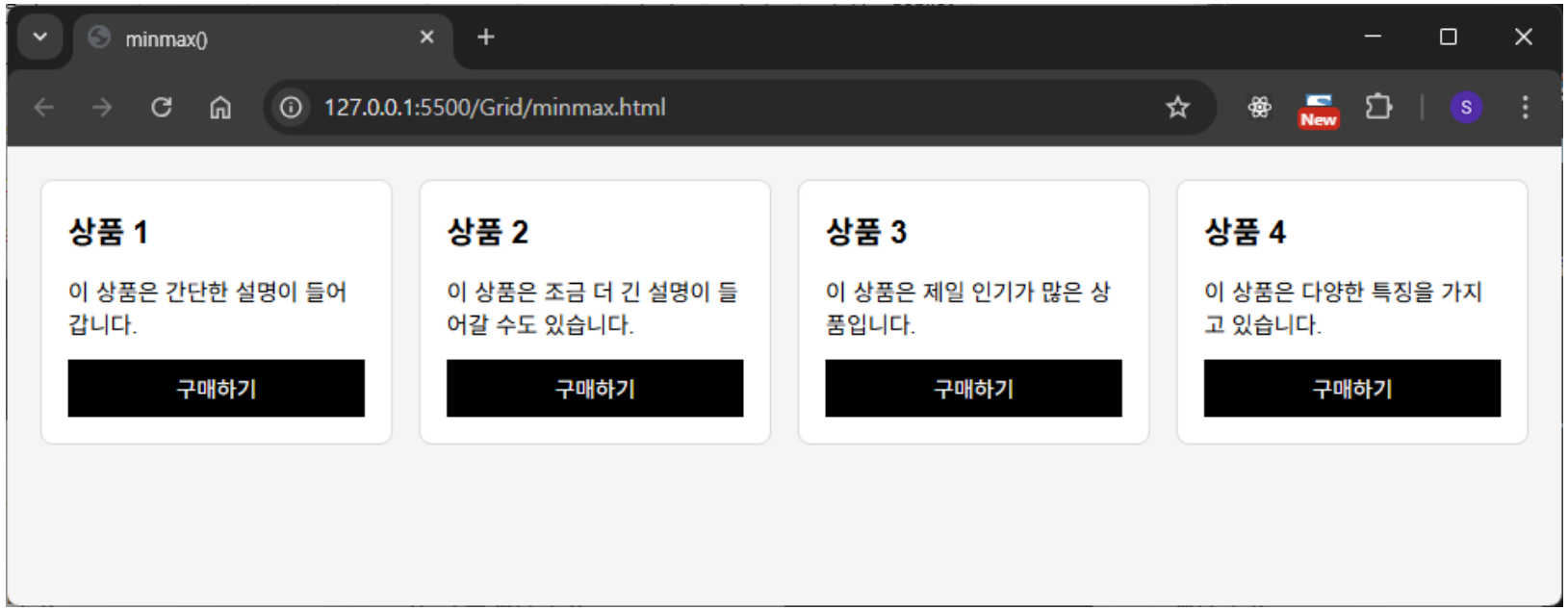
style.css

```
.container {
    border: 2px solid black;
    display: grid;
    grid-template-columns: minmax(200px, 1fr);
}

.container > div {
    border: 1px solid gray;
}
```



ⓒ2026.정수아.All rights reserved.

- **Grid 실습**
  - repeat() + minmax()
    - repeat(4, minmax(200px, 1fr))

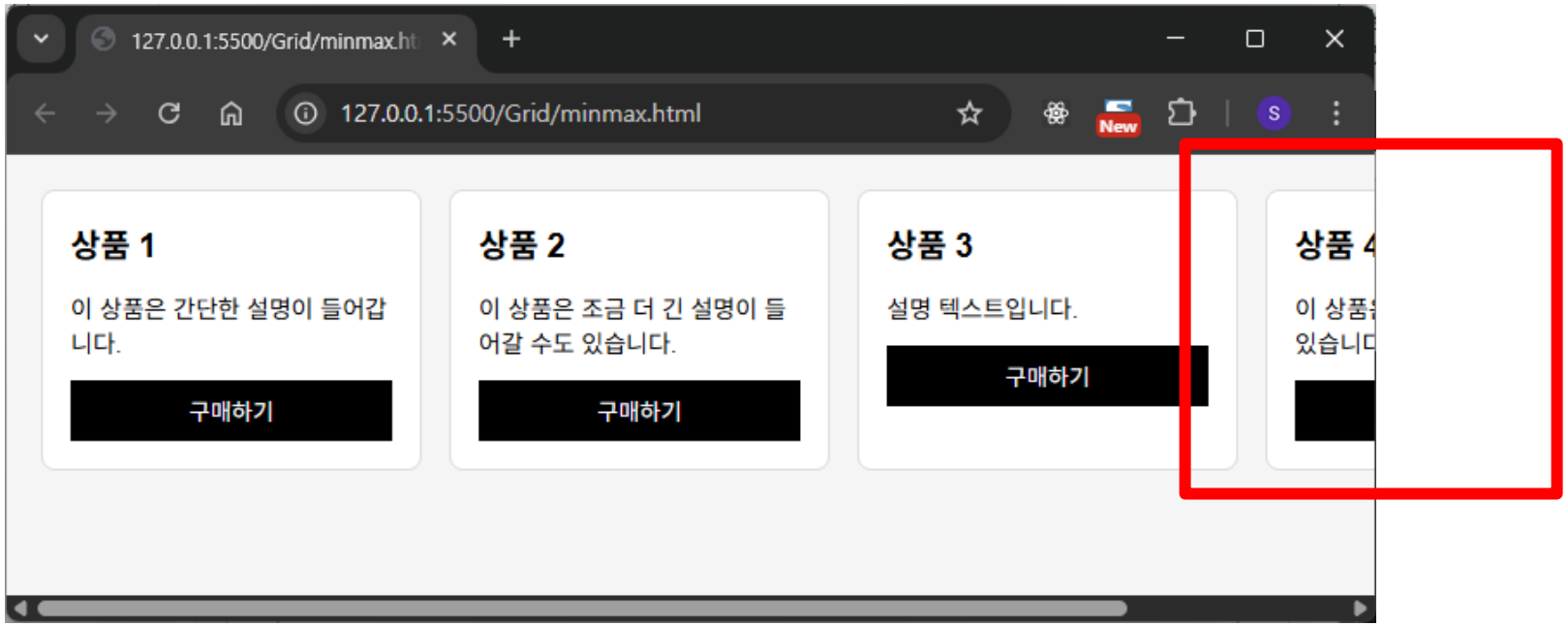


화면 크기가 큰 경우

ⓒ2026.정수아.All rights reserved.

Grid 기반 페이지 레이아웃 구현

- **Grid 실습**
  - repeat() + minmax()
    - repeat(4, minmax(200px, 1fr))

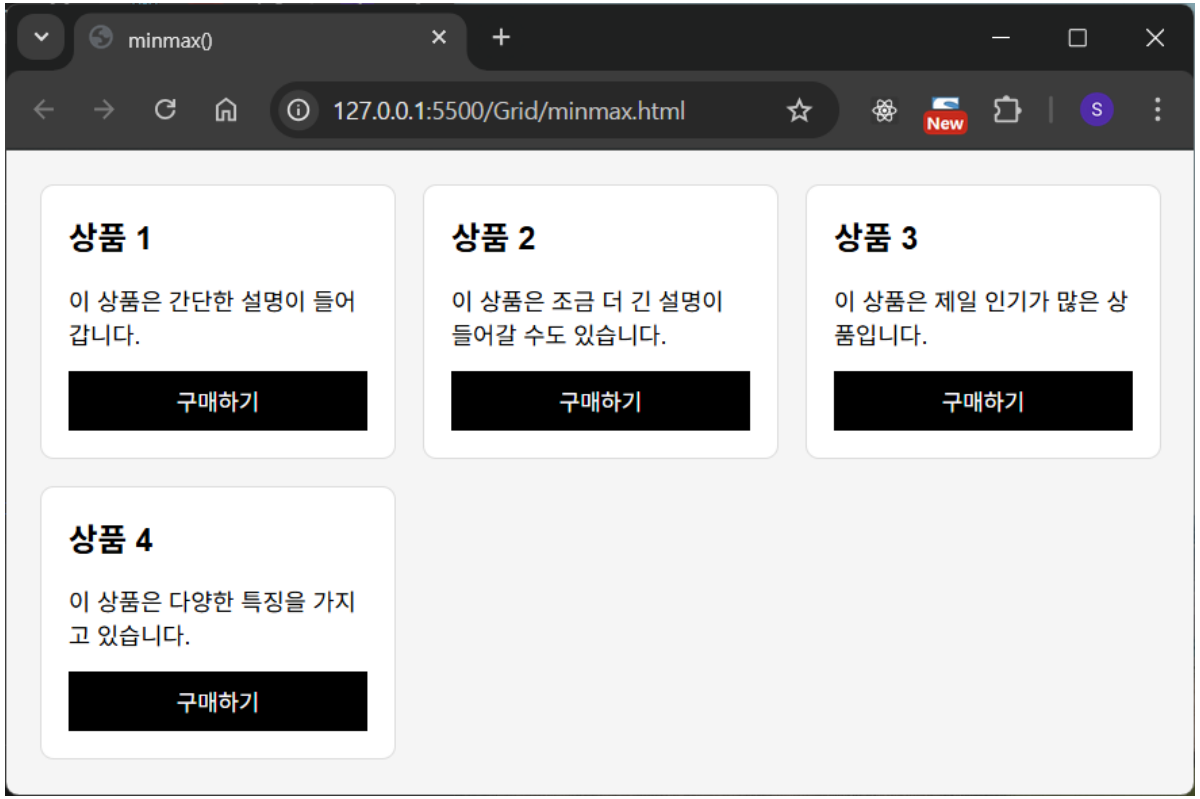


화면 크기가 작은 경우

ⓒ2026.정수아.All rights reserved.

Grid 기반 페이지 레이아웃 구현

- **Grid 실습**
  - repeat() + minmax()
    - repeat(auto-fit, minmax(200px, 1fr))



ⓒ2026.정수아.All rights reserved.

반응형 구조

- **반응형 웹(Responsive web)**
  - 다양한 화면 크기에 맞게 레이아웃이 변하는 웹
  - 하나의 코드로 모든 디바이스 대응
    - 모바일, 태블릿, 데스크탑
  - 필요한 이유
    - 화면이 작아지면 글자가 깨짐
    - 화면이 작아지면 레이아웃 붕괴

ⓒ2026.정수아.All rights reserved.

반응형 구조

- **반응형 웹(Responsive web)**
  - 만드는 방법
    - CSS 자체로 해결
      - flex
      - grid
      - minmax
      - auto-fit
    - 화면 크기에 따라 레이아웃을 다르게 적용
      - media query

ⓒ2026.정수아.All rights reserved.

- **반응형 웹(Responsive web)**
  - media query
    - 특정 조건에서만 CSS를 적용하는 기능

style.css

```
@media (조건) {

    /* 조건이 맞을 때 적용되는 스타일 */
}
```

```
@media (max-width: 768px) {
    body {
            background: red;
    }
}
```

ⓒ2026.정수아.All rights reserved.

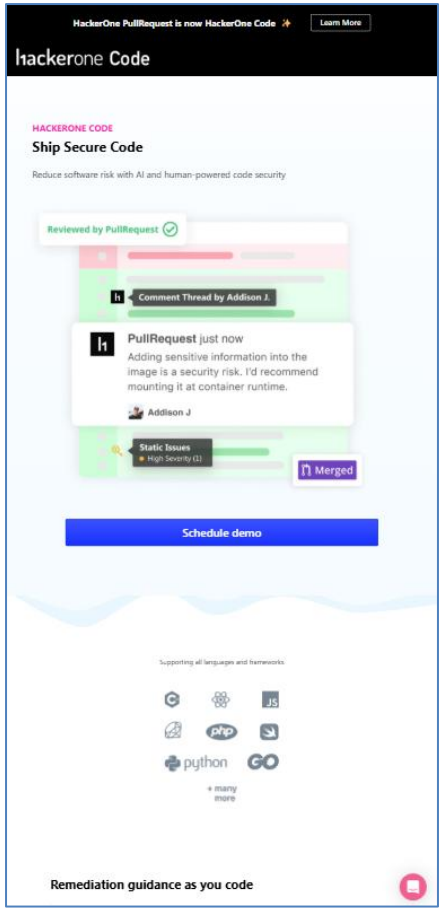
- **반응형 웹(Responsive web)**
  - 브레이크포인트(Breakpoint)
    - 레이아웃이 깨지기 시작하는 지점 기준

구분	기준
모바일	~767px
태블릿	768px~1022px
데스크탑	1024px~

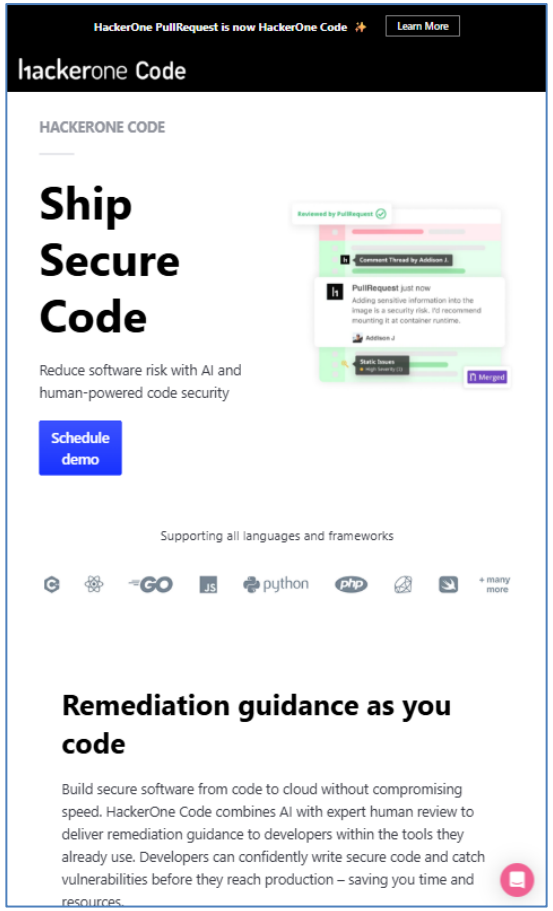
ⓒ2026.정수아.All rights reserved.

반응형 UI 적용

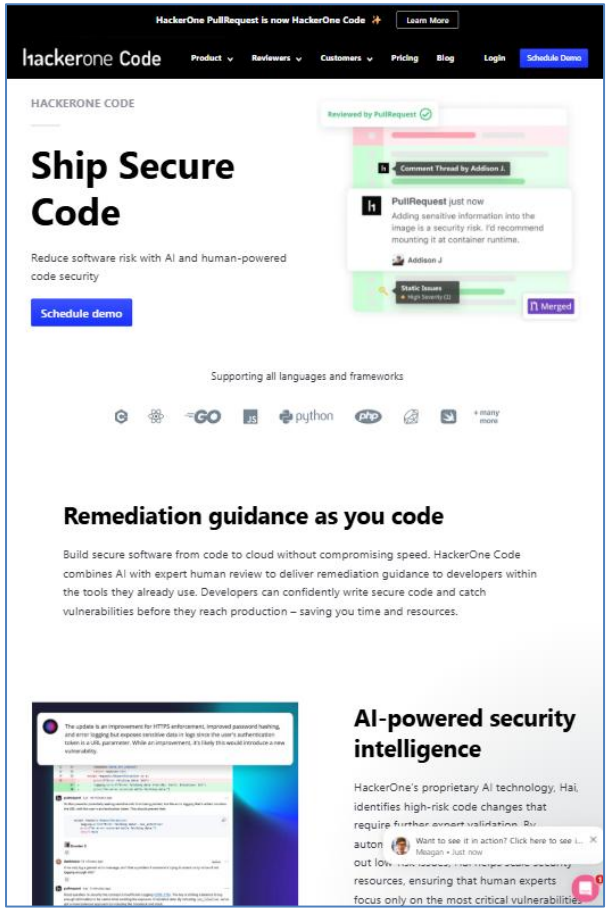
- **반응형 웹(Responsive web)**
  - https://www.pullrequest.com/



모바일



태블릿



PC

ⓒ2026.정수아.All rights reserved.

반응형 UI 적용

- **반응형 웹(Responsive web)**
  - 모든 웹사이트를 반응형으로 만들어야 할까?
    - 반응형이 필요한 경우
      - 다양한 기기에서 사용
      - 모바일 사용 비율 높음
      - 쇼핑몰, SNS, 포털 등
    - 반응형이 필요하지 않은 경우
      - 관리자 시스템
      - 특정 환경에서만 사용
      - 고정된 해상도

ⓒ2026.정수아.All rights reserved.

반응형 UI 적용

- **반응형 웹(Responsive web)**
  - media query
    - 화면 크기에 따라 다른 스타일을 적용하는 기능
    - 구조

```
@media (조건) {
    선택자 {
            스타일
    }
}
```

ⓒ2026.정수아.All rights reserved.

반응형 UI 적용

- **media query 실습**

768px 이상

768px 이하

ⓒ2026.정수아.All rights reserved.

CSS 프레임워크 활용 전략

- **CSS 프레임워크**
  - 미리 정의된 스타일과 구조를 제공하는 CSS 도구
  - 사용 이유
    - 개발 속도 향상
    - 일관된 디자인 유지
    - 유지보수 쉬움

ⓒ2026.정수아.All rights reserved.

- **CSS 프레임워크**
  - 종류

구분	특징	예시
컴포넌트 기반	완성된 UI 제공	Bootstrap
유틸리티 기반	클래스 조합으로 디자인	Tailwind
디자인 시스템 기반	일관된 디자인 규칙	Material UI

ⓒ2026.정수아.All rights reserved.

CSS 프레임워크 활용 전략

- **CSS 프레임워크**
  - 컴포넌트 기반(Bootstrap)
    - 버튼, 카드, 네비게이션 등 완성된 UI 제공
    - 빠르게 결과물 제작 가능
    - 디자인이 비슷해질 수 있음

ⓒ2026.정수아.All rights reserved.

CSS 프레임워크 활용 전략

- **CSS 프레임워크**
  - 유틸리티 기반(Tailwind)
    - 작은 CSS 클래스를 조합해서 UI 구성
    - 디자인 자유도 높음
    - 초기 학습 난이도 있음

ⓒ2026.정수아.All rights reserved.

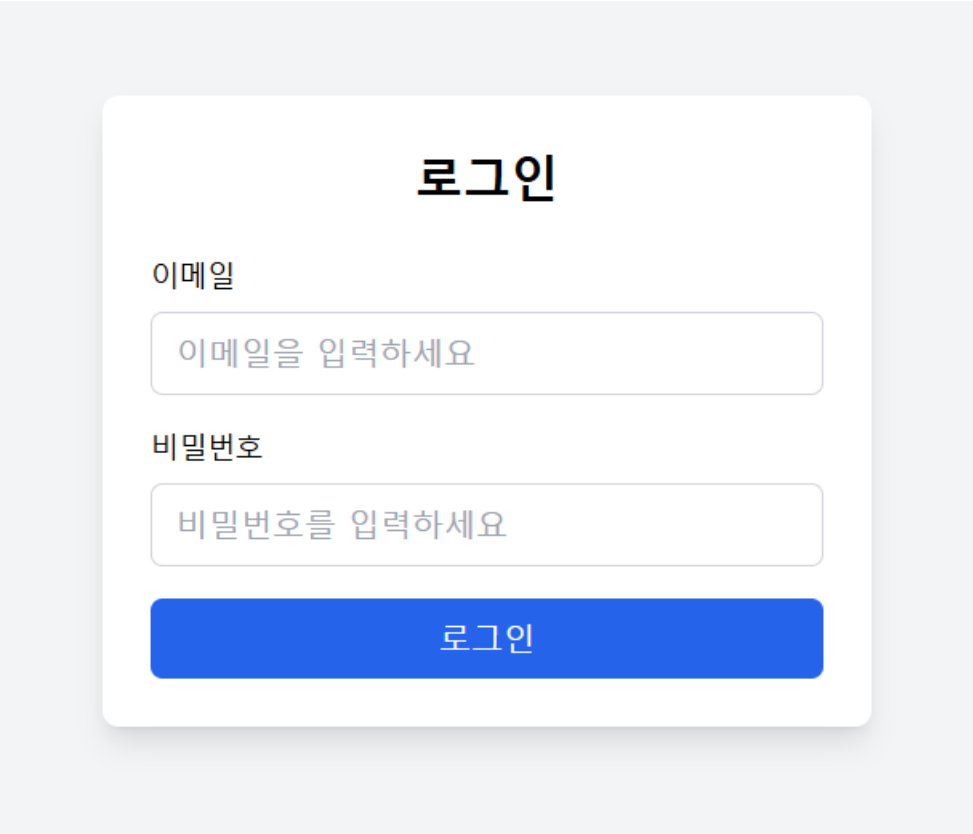
CSS 프레임워크 활용 전략

- **CSS 프레임워크**
  - 디자인 시스템 기반(Material UI)
    - 디자인 규칙 + UI 제공
    - 일관성 매우 높음
    - 자유도 제한

ⓒ2026.정수아.All rights reserved.

CSS 프레임워크 활용 전략

- **Bootstrap vs Tailwind**



ⓒ2026.정수아.All rights reserved.

- **Bootstrap 실습(login-bootstrap.html)**
  - Bootstrap 연결

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"/>
```

  - body 배경색

```
<body class="bg-light">
```

클래스	의미	색상 느낌
bg-primary	주요 색상	파란색 계열
bg-secondary	보조 색상	회색
bg-success	성공	초록
bg-danger	위험/에러	빨강
bg-info	정보	하늘색

ⓒ2026.정수아.All rights reserved.

- **Bootstrap 실습(login-bootstrap.html)**
  - 바깥쪽 전체 영역

```
<div class="container min-vh-100 d-flex justify-content-center align-items-center">
```

클래스	설명
container	페이지 내용을 감싸는 기본 레이아웃 클래스
min-vh-100	최소 높이를 화면 높이 기준으로 설정하는 클래스
d-flex	display:flex를 적용하는 클래스
justify-content-center	주축 방향으로 가운데 정렬
align-items-center	교차축 방향으로 가운데 정렬

ⓒ2026.정수아.All rights reserved.

- **Bootstrap 실습(login-bootstrap.html)**
  - 로그인 카드 박스

```
<div class="card shadow p-4" style="width: 100%; max-width: 400px;">
```

클래스	설명
card	카드 컴포넌트 클래스
shadow	박스에 그림자를 적용하는 클래스
p-4	요소 내부에 padding을 적용하는 클래스
width: 100%	부모 영역 안에서 width를 최대한으로 사용
max-width: 400px	400px보다는 넓어지지 않음

ⓒ2026.정수아.All rights reserved.

- **Bootstrap 실습(login-bootstrap.html)**
  - 제목

```
<h2 class="text-center mb-4">로그인</h2>
```

클래스	설명
text-center	텍스트를 가운데 정렬하는 클래스
mb-4	요소의 아래쪽에 margin을 적용하는 클래스

ⓒ2026.정수아.All rights reserved.

- **Bootstrap 실습(login-bootstrap.html)**
  - 이메일 및 비밀번호 입력 영역

```
<div class="mb-3">
   <label for="email" class="form-label">이메일</label>
   <input type="email" id="email" class="form-control" placeholder="이메일을 입력하세요">
</div>

<div class="mb-3">
  <label for="password" class="form-label">비밀번호</label>
  <input type="password" id="password" class="form-control" placeholder="비밀번호를 입력하세요">
</div>
```

클래스	설명
mb-3	요소의 아래쪽에 margin을 적용하는 클래스
form-label	form에서 입력 요소의 label을 스타일링하는 클래스
form-control	form에서 입력 요소를 스타일링하는 클래스

ⓒ2026.정수아.All rights reserved.

- **Bootstrap 실습(login-bootstrap.html)**
  - 로그인 버튼

```
<button class="btn btn-primary w-100">로그인</button>
```

클래스	설명
btn	버튼의 기본 클래스
btn-primary	primary 색상 스타일이 적용된 버튼 클래스
w-100	width를 100%로 설정

ⓒ2026.정수아.All rights reserved.

- **Tailwind 실습(login-tailwind.html)**
  - Tailwind 연결

```
<script src="https://cdn.tailwindcss.com"></script>
```

  - body 배경색

```
<body class="bg-gray-100">
```

숫자 값	의미	숫자 값	의미
50	거의 흰색	600~700	진해짐
100	매우 밝음	800~900	매우 어두움
200~400	밝은 영역	950	거의 검정
500	기본 색		

ⓒ2026.정수아.All rights reserved.

- **Tailwind 실습(login-tailwind.html)**
  - 바깥쪽 전체 영역

```
<div class="min-h-screen flex items-center justify-center">
```

클래스	설명
min-h-screen	최소 높이를 화면 전체 높이 만큼 설정
flex	display:flex를 적용하는 클래스
items-center	세로 방향 가운데 정렬
justify-center	가로 방향 가운데 정렬

ⓒ2026.정수아.All rights reserved.

- **Tailwind 실습(login-tailwind.html)**
  - 로그인 카드 박스

```
<div class="w-full max-w-sm bg-white shadow-lg rounded-lg p-6">
```

클래스	설명
w-full	요소의 너비를 부모의 100%로 만드는 클래스
max-w-sm	요소의 최대 너비를 sm 크기로 제한하는 클래스
bg-white	배경을 흰색으로 지정
shadow-lg	큰 그림자를 적용하는 클래스
rounded-lg	모서리를 둥글게 만드는 클래스
p-6	요소 내부에 padding을 적용하는 클래스

ⓒ2026.정수아.All rights reserved.

- **Tailwind 실습(login-tailwind.html)**
  - 제목

```
<h2 class="text-2xl font-bold text-center mb-6">로그인</h2>
```

클래스	설명
text-2xl	텍스트 크기를 크게 만드는 클래스
font-bold	텍스트 굵기를 두껍게 설정하는 클래스
text-center	텍스트를 가운데 정렬하는 클래스
mb-6	요소 아래쪽에 margin을 적용하는 클래스

ⓒ2026.정수아.All rights reserved.

- **Tailwind 실습(login-tailwind.html)**
  - 이메일 및 비밀번호 입력 영역

```
<div class="mb-4">
  <label for="email" class="block text-sm font-medium mb-2">이메일</label>
  <input type="email" id="email" placeholder="이메일을 입력하세요"
    class="w-full border border-gray-300 rounded-md px-3 py-2">
</div>

<div class="mb-4">
        <label for="password" class="block text-sm font-medium mb-2">비밀번호</label>
        <input type="password" id="password" placeholder="비밀번호를 입력하세요"
          class="w-full border border-gray-300 rounded-md px-3 py-2">
</div>
```

클래스	설명
block	label을 block 요소로 변경
px-3	요소의 좌우 padding에 여백을 적용하는 클래스
py-2	요소의 상하 padding에 여백을 적용하는 클래스

ⓒ2026.정수아.All rights reserved.

- **Tailwind 실습(login-tailwind.html)**
  - 로그인 버튼

```
<button class="w-full bg-blue-600 text-white py-2 rounded-md">로그인</button>
```

클래스	설명
text-white	버튼의 글자 색을 흰색으로 지정
rounded-md	모서리를 둥글게 만드는 클래스

ⓒ2026.정수아.All rights reserved.

CSS 프레임워크 활용 전략

- **Bootstrap, Tailwind vs Material UI**

구분	Bootstrap, Tailwind	Material UI
구조	HTML + CSS	React 컴포넌트
방식	클래스 조합	컴포넌트 사용
자유도	높음	규칙 기반
난이도	낮음	상대적으로 높음

ⓒ2026.정수아.All rights reserved.

- **Q&A**
  - 실무에서는 어떤 프레임워크를 가장 많이 써요?
  - 어떤 것을 배우는게 좋을까요?
  - 프레임워크를 안 쓰고 CSS만 쓰면 안 되나요?

ⓒ2026.정수아.All rights reserved.

UI 최적화 및 리팩토링

- **UI 최적화**
  - 사용자 경험 개선
  - 성능 개선
  - 불필요한 요소 제거

- **리팩토링**
  - 기능은 그대로 유지
  - 코드 구조 개선
  - 유지보수 용이

ⓒ2026.정수아.All rights reserved.

- **리팩토링 예시**

  - 나쁜 예

```
<button style="background: blue; padding: 10px;">확인</button>
<button style="background: blue; padding: 10px;">저장</button>
<button style="background: blue; padding: 10px;">전송</button>
```

  - 좋은 예

```
<button class="btn">확인</button>
<button class="btn">저장</button>
<button class="btn">전송</button>
```

```
.btn {
    background: blue;
    padding: 10px;
}
```

ⓒ2026.정수아.All rights reserved.

- **코드가 사용자 경험에 미치는 영향**
  - 코드 구조 → 사용자 경험
    - 일관성 → 사용자의 신뢰감
    - 빠른 수정 → 오류 감소
    - 구조 명확 → 예측 가능성 증가
    - 재사용 가능 → UI 안정성 유지

ⓒ2026.정수아.All rights reserved.

- **UI 최적화와 체크리스트**
  - 간격이 일정한가?
  - 버튼 크기가 적절한가?
  - 클릭 영역이 충분한가?
  - 정보가 너무 많지 않은가?
  - 반응 속도가 빠른가?

ⓒ2026.정수아.All rights reserved.

- **CSS 관리의 한계**
  - 스타일 반복 증가
  - 수정 비용 증가
  - 코드 복잡도 증가

ⓒ2026.정수아.All rights reserved.

UI 최적화 및 리팩토링

- **해결 방법 : Sass**
  - CSS를 더 효율적으로 관리하는 도구
  - 반복되는 스타일을 줄여줌
  - 값을 한 곳에서 관리 가능

ⓒ2026.정수아.All rights reserved.

- **Sass**
  - 작성 방식

구분	Sass(옛날 방식)	Scss(현재 방식)
문법	들여쓰기 기반	CSS와 동일
중괄호	없음	있음
세미콜론	없음	있음
사용	거의 사용하지 않음	대부분 사용

ⓒ2026.정수아.All rights reserved.

- **Sass**
  - 변수(Variable)
    - 값을 한 곳에서 관리
    - 수정 시 전체 반영

```
.button {
    background: blue;
    color: white;
}

.card {
    border: 2px solid blue;
}

.title {
    color: blue;
}
```

CSS

```
$main-color: blue;

.button {
    background: $main-color;
    color: white;
}

.card {
    border: 2px solid $main-color;
}

.title {
    color: $main-color;
}
```

SCSS

ⓒ2026.정수아.All rights reserved.

- **Sass**
  - 중첩(Nesting)
    - 구조를 계층적으로 표현
    - 가독성 향상

```
.card {
    padding: 20px;
}

.card .title {
    font-size: 20px;
}

.card .button {
    background: blue;
}
```

CSS

```
.card {
    padding: 20px;

    .title {
      font-size: 20px;
    }

    .button {
      background: blue;
    }
}
```

SCSS

ⓒ2026.정수아.All rights reserved.

- **구조를 이름으로 표현하는 방법 : BEM**
  - BEM(Block, Element, Modifier)
  - 구조를 이름으로 표현하는 방식
  - 역할과 관계가 이름에 드러남
  - 코드의 가독성과 유지보수 향상

ⓒ2026.정수아.All rights reserved.

- **BEM(Block, Element, Modifier)**
  - 구조
    - Block
      - 하나의 독립된 UI 단위
      - card, button 등
    - Element
      - Block 내부에 속한 요소
      - card 안의 제목(card__title), card 안의 버튼(card__button) 등
    - Modifier
      - 상태 또는 변형을 표현
      - 기본 버튼, 강조 버튼, 비활성 버튼 등
      - button button—primary

ⓒ2026.정수아.All rights reserved.

- **BEM(Block, Element, Modifier)**
  - 예시

```
<div class="card">
  <h2 class="card__title">제목</h2>
  <button class="card__button button button--primary">확인</button>
  <button class="card__button button button--disabled">취소</button>
</div>
```

  - Block
    - card, button
  - Element
    - card__title, card__button
  - Modifier
    - button--primary, button—disabled

ⓒ2026.정수아.All rights reserved.

- **Sass**
  - Mixin
    - 여러 줄의 스타일을 하나로 묶어서 이름을 붙이고 필요할 때 가져다 쓰는 기능

```
.button {
    padding: 10px;
    border-radius: 5px;
}

.card-button {
    padding: 10px;
    border-radius: 5px;
}
```

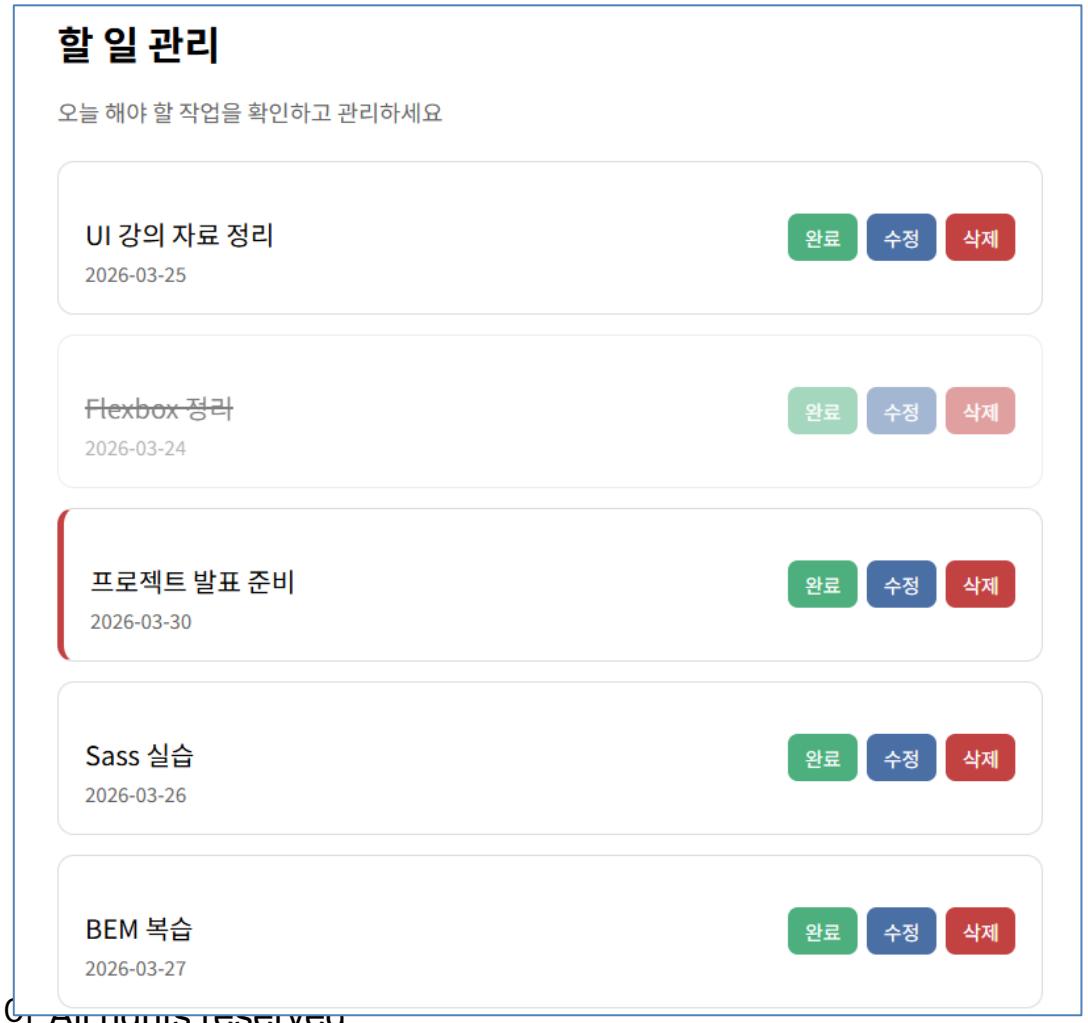
CSS

```
@mixin button-style {
    padding: 10px;
    border-radius: 5px;
}

.button {
    @include button-style;
}

.card-button {
    @include button-style;
}
```

SCSS

ⓒ2026.정수아.All rights reserved.

- **Sass + BEM 코드 분석**



Block : todo
Element : todo__title, todo__button
Modifier : todo--done, todo--important

변수 : $primary, $gray, $danger, $success
중첩 : todo → todo__content
→ todo__title
→ todo__date
Mixin : button-base, button-color

ⓒ2026.정수아.All rights reserved.

정리

사용자 이해
- UX 핵심 원칙
- 사용자 중심 사고

→

구조 설계
- UI/UX 설계
- 컴포넌트 구조 분석

→

레이아웃 구성
- Flexbox
- Grid

↓

확장 대응
- 반응형 UI

←

실무 적용
- CSS 프레임워크

←

코드 구조화
- Sass
- BEM

→

UI 최적화 및 리팩토링

ⓒ2026.정수아.All rights reserved.

수고하셨습니다!

ⓒ2026.정수아.All rights reserved.